



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**PROYECTO "APLICACIÓN MÓVIL COLABORATIVA PARA MEJORAR
LA EFECTIVIDAD EN LA ASISTENCIA DE MASCOTAS PERDIDAS
CON PARTICIPACIÓN DE VOLUNTARIOS EN LA CIUDAD DE TACNA,
2026"**

Curso: Construcción de software II

Docente: RICARDO EDUARDO VALCARCEL ALVARADO

Integrantes:

Chite Quispe Brian Danilo

(2021070015)

Anampa Pancca David Jordan

(2022074268)

Tacna – Perú

2026



Índice

Introducción	3
Laboratorio N° 01 – REVISION DE CODIGO	4
1. Información sobre el evento práctico	4
Objetivo general	4
1. Desarrollo de la Práctica	5
Qué se observa en el código	5
1) Repetición de providers	5
2) Repetición de pantalla de login	6
3) Repetición de notificaciones	6
Por qué es un problema	20
Qué está bien	22
No te repitas	26
Comenta lo necesario.....	26
Fail Fast.....	26
Evita los números mágicos.....	26
Utiliza buenos nombres.....	26
No uses variables globales	26
Devuelve valores, no los imprimas	26
Utiliza los espacios en blanco para mejor lectura.....	27
No te repitas	30
Comenta lo necesario.....	30
Fail Fast.....	30
Evita los números mágicos.....	31
Utiliza buenos nombres.....	31
No uses variables globales	31
Devuelve valores, no los imprimas	31
Utiliza los espacios en blanco para mejor lectura.....	31
4. Auditoría asistida con Antigraity: aplicación de las 9 reglas	60

Introducción

La calidad del código constituye un aspecto fundamental en el desarrollo de software, debido a que influye directamente en la mantenibilidad, legibilidad, escalabilidad y confiabilidad de una aplicación. A medida que un proyecto crece, la ausencia de estándares y buenas prácticas puede generar problemas como duplicación de código, dificultad para realizar modificaciones, errores recurrentes y una mayor complejidad en el mantenimiento del sistema.

En el presente laboratorio se realiza una revisión y refactorización del código desarrollado para el proyecto “Aplicación móvil colaborativa para mejorar la efectividad en la asistencia de mascotas perdidas con participación de voluntarios en la ciudad de Tacna, 2026”, desarrollado en el curso de Construcción de Software II. La revisión se enfoca principalmente en los módulos relacionados con la autenticación de usuarios, el registro y visualización de avistamientos, y la gestión de comentarios en las publicaciones.

Para esta evaluación se consideran principios y buenas prácticas de construcción de software, tales como evitar la duplicación de código (DRY), utilizar comentarios únicamente cuando aporten información relevante, aplicar el principio Fail Fast, evitar números mágicos, emplear nombres descriptivos, reducir el uso de variables globales, retornar resultados en lugar de utilizar impresiones por consola y organizar adecuadamente los espacios en blanco. Asimismo, se revisan aspectos relacionados con las convenciones de nomenclatura y la responsabilidad de las variables y métodos.

A partir de los problemas identificados en el código original, se realizaron diferentes acciones de refactorización, entre ellas la creación de utilidades reutilizables, la centralización de procesos repetitivos, la incorporación de constantes descriptivas, la validación temprana de datos y archivos, la mejora de los nombres de variables y la reorganización del acceso a los datos almacenados en Firebase. Estas modificaciones buscan obtener un código más limpio, comprensible y fácil de mantener.

Adicionalmente, se empleó el agente de IA Antigravity como auditor asistido; sus resultados y las observaciones del equipo se documentan en la sección 4.

Finalmente, la práctica permite comprobar la importancia de realizar revisiones periódicas del código durante el desarrollo de un proyecto, ya que estas permiten detectar deficiencias antes de que se conviertan en problemas mayores y contribuyen a construir una aplicación más ordenada, estable y preparada para futuras mejoras.

Laboratorio N° 01 – REVISION DE CODIGO

1. Información sobre el evento práctico

1.1. Objetivos:

Objetivo general

- Realizar la revisión del código fuente de una aplicación de software, considerando buenas prácticas y estándares de construcción de software, con la finalidad de identificar errores, mejorar la calidad del código y facilitar su mantenimiento.

Objetivos específicos

- Identificar errores y posibles problemas en el código fuente.
- Verificar el cumplimiento de estándares y convenciones de programación.
- Evaluar la estructura, organización y legibilidad del código.
- Detectar código duplicado, innecesario o difícil de mantener.
- Comprobar el correcto uso de nombres para variables, métodos, clases y componentes.
- Proponer mejoras que permitan obtener un código más limpio, seguro y mantenible.
- Documentar las observaciones y recomendaciones encontradas durante la revisión.

1.2. Equipos, materiales, programas y recursos

- Computadora o laptop.
- Conexión a Internet, de ser necesaria.
- Software ofimático para la elaboración de la documentación.
- Software requerido para su aplicación
- Aplicación diseñada e implementada en el Curso de Construcción de Software II
- Formatos de despliegue para la documentación del Proyecto de Software en proceso de implementación

1.3. Seguridad

- Protocolo de Seguridad para laboratorios y talleres
- Protocolo ante la presencialidad en Laboratorios

1. Desarrollo de la Práctica

• Autenticar Usuario

- No te repitas.

En el código de autenticación, esta parte se ve claramente en la ruta principal de app.dart:, donde hay muchas entradas repetidas con la misma estructura:

- cada ruta crea un provider
- cada ruta asigna una pantalla
- se repite el patrón de:
 - ChangeNotifierProvider(
 - create: (_) => XVM(),
 - child: const PantallaX(),

Eso es exactamente duplicación de código.

Qué se observa en el código

1) Repetición de providers

En varias rutas se repite un patrón similar:

- /login
- /registro
- /recuperar
- /perfil
- /inicio

Todos siguen la misma forma:

- crear un ViewModel
- enlazarlo con la pantalla
- devolver la pantalla

Eso se puede centralizar.

2) Repetición de pantalla de login

También aparece aquí:

- `"/login"` crea `LoginVM`
- `"/inicio"` también crea `LoginVM` dentro de `loginScreen`

Esto significa que la misma lógica se está creando más de una vez.

3) Repetición de notificaciones

También se repite esta parte:

- `"/inicio"` usa `NotificacionVM().escucharNotificaciones()`
- `"/notificaciones"` usa `NotificacionVM().escucharNotificaciones()`

Entonces el mismo patrón se construye dos veces.

```
71      }, // ThemeData
72      initialRoute: "/login",
73      routes: {
74        "/login": (_) => ChangeNotifierProvider(
75          create: (_) => LoginVM(),
76          child: const PantallaLogin(),
77        ), // ChangeNotifierProvider
78        "/registro": (_) => ChangeNotifierProvider(
79          create: (_) => RegistroVM(),
80          //create: (_) => RegistroVM(apiDni: api),
81          child: const PantallaRegistro(),
82        ), // ChangeNotifierProvider
83        "/registroAdmin": (_) => ChangeNotifierProvider(
84          create: (_) => AdminVM(apiDni: api),
85          child: const PantallaRegistroAdmin(),
86        ), // ChangeNotifierProvider
87        "/recuperar": (_) => ChangeNotifierProvider(
88          create: (_) => RecuperarVM(),
89          child: const PantallaRecuperar(),
90        ), // ChangeNotifierProvider
91        "/verificaEmail": (_) => const PantallaVerificaEmail(),
92        "/perfil": (_) => ChangeNotifierProvider(
93          create: (_) => PerfilVM(),
94          child: const PantallaPerfil(),
95        ), // ChangeNotifierProvider
96        "/inicio": (_) => AuthWrapper(
97          loginScreen: ChangeNotifierProvider(
98            create: (_) => LoginVM(),
99            child: const PantallaLogin(),
```

La parte:

- `"/inicioAdmin": (_) => const PantallaInicioAdmin(),`
- `"/adminUsuarios": (_) => const PantallaAdminUsuarios(),`
- `"/adminReportes": (_) => const PantallaAdminReporte(),`
- `"/adminNotificaciones": (_) => ChangeNotifierProvider(...)`

muestra que el patrón se repite:

- una ruta
- una pantalla
- o un provider + pantalla

```
100 .....), // ChangeNotifierProvider
101 .....homeScreenBuilder: (uid, userData) => ChangeNotifierProvider(
102 .....  create: (_) => NotificacionVM().escucharNotificaciones(),
103 .....  child: const PantallaInicio(),
104 .....), // ChangeNotifierProvider
105 .....), // AuthWrapper
106 .....
107 ....."/inicioAdmin": (_) => const PantallaInicioAdmin(),
108 ....."/adminUsuarios": (_) => const PantallaAdminUsuarios(),
109 ....."/adminReportes": (_) => const PantallaAdminReporte(),
110 ....."/adminNotificaciones": (_) => ChangeNotifierProvider(
111 .....  create: (_) => AdminNotificacionesVM(),
112 .....  child: const PantallaAdminNotificaciones(),
113 .....), // ChangeNotifierProvider
```

La configuración de rutas y proveedores en la clase `MyApp` reutiliza el mismo patrón para distintos accesos, como login, registro y recuperación de contraseña, lo que incrementa la longitud del código y dificulta su mantenimiento.

En `auth_wrapper.dart`, el flujo de autenticación hace varias validaciones con patrones parecidos:

- verificar si existe usuario
- verificar si el documento existe
- verificar si los datos del usuario no son nulos
- verificar si la cuenta está activa
- cerrar sesión y devolver login

Es decir, el flujo se repite en varias ramas:

- si no hay usuario autenticado

- si el documento no existe
- si el userData es null
- si la cuenta está desactivada

Cada caso hace algo parecido:

- imprimir un mensaje
- cerrar sesión
- volver al login

Eso sí es duplicación de lógica.

```
31 ..... stream: FirebaseAuth.instance.authStateChanges(),
32 ..... builder: (context, authSnapshot) {
33 ..... // Si no hay usuario autenticado, mostrar login
34 ..... if (!authSnapshot.hasData || authSnapshot.data == null) {
35 .....   print('🔒 No hay usuario autenticado - Mostrando login');
36 .....   return loginScreen;
37 ..... }
38 .....
39 ..... final user = authSnapshot.data!;
40 .....
41 ..... // Escuchar cambios en el documento del usuario en tiempo real
42 ..... return StreamBuilder<DocumentSnapshot>(
43 .....   stream: FirebaseFirestore.instance
44 .....     .collection('usuarios')
45 .....     .doc(user.uid)
46 .....     .snapshots(),
47 .....   builder: (context, userSnapshot) {
48 .....     // Mientras carga, mostrar indicador
49 .....     if (userSnapshot.connectionState == ConnectionState.waiting) {
50 .....       return const Scaffold(
51 .....         backgroundColor: Colors.white,
52 .....         body: Center(
53 .....           child: Column(
54 .....             mainAxisAlignment: MainAxisAlignment.center,
55 .....             children: [
56 .....               CircularProgressIndicator(),
57 .....               SizedBox(height: 16),
58 .....               Text(
59 .....                 'Verificando cuenta...',
60 .....                 style: TextStyle(
```

```

59 ..... 'Verificando cuenta...',
60 ..... style: TextStyle(
61 .....   fontSize: 16,
62 .....   color: Colors.grey,
63 ..... ), // TextStyle
64 ..... ), // Text
65 ..... ],
66 ..... ), // Column
67 ..... ), // Center
68 ..... ); // Scaffold
69 ..... }
70 .....
71 ..... // ✖ Si hay error o no existe el documento
72 ..... if (userSnapshot.hasError ||
73 .....   !userSnapshot.hasData ||
74 .....   !userSnapshot.data!.exists) {
75 .....   print('⚠ Error o documento no existe para usuario: ${user.uid}');
76 .....   // Cerrar sesión y volver al login
77 .....   WidgetsBinding.instance.addPostFrameCallback((_) async {
78 .....     await FirebaseAuth.instance.signOut();
79 .....     if (context.mounted) {
80 .....       ScaffoldMessenger.of(context).showSnackBar(
81 .....         SnackBar(
82 .....           content: const Row(
83 .....             children: [
84 .....               Icon(Icons.error_outline, color: Colors.white),
85 .....               SizedBox(width: 12),
86 .....               Expanded(
87 .....                 child: Text(
88 .....                   'Error al cargar los datos del usuario.'

```

```

88 .....                   'Error al cargar los datos del usuario.',
89 .....                   style: TextStyle(fontSize: 15),
90 .....                 ), // Text
91 .....               ), // Expanded
92 .....             ],
93 .....           ), // Row
94 .....           backgroundColor: Colors.red,
95 .....           duration: const Duration(seconds: 4),
96 .....           behavior: SnackBarBehavior.floating,
97 .....           shape: RoundedRectangleBorder(
98 .....             borderRadius: BorderRadius.circular(10),
99 .....           ), // RoundedRectangleBorder
100 .....         ), // SnackBar
101 .....       );
102 .....     }
103 .....   });
104 .....   return loginScreen;
105 ..... }
106 .....
107 ..... // 📦 Obtener datos del usuario
108 ..... final userData = userSnapshot.data!.data() as Map<String, dynamic>;
109 .....
110 ..... if (userData == null) {
111 .....   print('⚠ userData es null para usuario: ${user.uid}');
112 .....   // Si no hay datos, cerrar sesión
113 .....   WidgetsBinding.instance.addPostFrameCallback((_) async {
114 .....     await FirebaseAuth.instance.signOut();
115 .....   });
116 .....   return loginScreen;
117 ..... }

```

```
119 ..... // ✅ Verificar si la cuenta está activa
120 ..... final bool isActive = userData['activo'] ?? true;
121 .....
122 ..... print('👤 Usuario: ${user.email}');
123 ..... print('📄 Estado activo: $isActive');
124 ..... print('👤 Rol: ${userData['rol']}');
125 .....
126 ..... // 🚫 Si la cuenta está DESACTIVADA, cerrar sesión
127 ..... if (!isActive) {
128 .....   print('❌ Cuenta DESACTIVADA - Cerrando sesión');
129 .....   WidgetsBinding.instance.addPostFrameCallback((_) async {
130 .....     await FirebaseAuth.instance.signOut();
131 .....     if (context.mounted) {
132 .....       ScaffoldMessenger.of(context).showSnackBar(
133 .....         SnackBar(
134 .....           content: const Row(
135 .....             children: [
136 .....               Icon(Icons.block, color: Colors.white, size: 28),
137 .....               SizedBox(width: 12),
138 .....               Expanded(
139 .....                 child: Column(
140 .....                   crossAxisAlignment: CrossAxisAlignment.start,
141 .....                   mainAxisAlignment: MainAxisAlignment.min,
142 .....                   children: [
143 .....                     Text(
144 .....                       'Cuenta Desactivada',
145 .....                       style: TextStyle(
146 .....                         fontSize: 16,
147 .....                         fontWeight: FontWeight.bold,
148 .....                       ), // TextStyle
149 .....                     ), // Text
150 .....                     SizedBox(height: 4),
151 .....                     Text(
152 .....                       'Tu cuenta ha sido desactivada. Contacta al administrador para más i
153 .....                       style: TextStyle(fontSize: 14),
154 .....                     ), // Text
155 .....                   ],
156 .....                 ), // Column
157 .....               ), // Expanded
158 .....             ],
159 .....           ), // Row
160 .....           backgroundColor: Colors.red.shade700,
161 .....           duration: const Duration(seconds: 6),
162 .....           behavior: SnackBarBehavior.floating,
163 .....           shape: RoundedRectangleBorder(
164 .....             borderRadius: BorderRadius.circular(12),
165 .....           ), // RoundedRectangleBorder
166 .....           margin: const EdgeInsets.all(16),
167 .....         ), // SnackBar
168 .....       );
169 .....     }
170 .....   });
171 .....   return loginScreen;
```

Asimismo, en AuthWrapper se repite la lógica de validación de sesión, usuario activo y redirección al login en varios bloques. Esta duplicidad afecta la legibilidad del código y puede provocar inconsistencias si se modifica una validación sin actualizar el resto.

- Comenta lo necesario.

En auth_wrapper.dart

Los comentarios iniciales sí son útiles porque explican:

- qué hace AuthWrapper
- qué valida
- cuándo bloquea acceso

Eso es un comentario necesario.

```
1 import 'package:flutter/material.dart';
2 import 'package:firebase_auth/firebase_auth.dart';
3 import 'package:cloud_firestore/cloud_firestore.dart';
4
5 /// AuthWrapper - Valida el estado de la cuenta en tiempo real
6 ///
7 /// Este wrapper monitorea constantemente:
8 /// - Si el usuario está autenticado
9 /// - Si la cuenta está activa (campo 'activo' en Firestore)
10 /// - Si el documento del usuario existe
11 ///
12 /// Si la cuenta es desactivada por un admin, el usuario será
13 /// expulsado automáticamente de la aplicación.
14 class AuthWrapper extends StatelessWidget {
15   /// Pantalla que se muestra cuando NO hay usuario autenticado
16   final Widget loginScreen;
17
18   /// Constructor que recibe el UID y los datos del usuario
19   /// para determinar qué pantalla mostrar
```

Pero también hay otros que son más repetitivos:

- “Si no hay usuario autenticado, mostrar login”
- “Escuchar cambios en el documento del usuario en tiempo real”
- “Mientras carga, mostrar indicador”
- “Si hay error o no existe el documento”
- “Verificar si la cuenta está activa”

```
28  @override
29  ✓  Widget build(BuildContext context) {
30  ✓    return StreamBuilder<User?>(
31      stream: FirebaseAuth.instance.authStateChanges(),
32  ✓    builder: (context, authSnapshot) {
33        // 🚫 Si no hay usuario autenticado, mostrar login
34  ✓    if (!authSnapshot.hasData || authSnapshot.data == null) {
35        print('🔒 No hay usuario autenticado - Mostrando login');
36        return loginScreen;
37    }
38  }
```

```
40
41      // 🕒 Escuchar cambios en el documento del usuario en tiempo real
42  ✓  return StreamBuilder<DocumentSnapshot>(
43      stream: FirebaseFirestore.instance
44          .collection('usuarios')
45          .doc(user.uid)
46          .snapshots(),
47  ✓    builder: (context, userSnapshot) {
48        // ⌚ Mientras carga, mostrar indicador
49        if (userSnapshot.connectionState == ConnectionState.waiting) {
50  ✓    return const Scaffold(
51        backgroundColor: Colors.white,
52  ✓    body: Center(
```

```
47    builder: (context, userSnapshot) {
48        // ⌚ Mientras carga, mostrar indicador
49        if (userSnapshot.connectionState == ConnectionState.waiting) {
50        return const Scaffold(
51        backgroundColor: Colors.white,
52        body: Center(
53            child: Column(
54                mainAxisAlignment: MainAxisAlignment.center,
55                children: [
56                    CircularProgressIndicator(),
57                    SizedBox(height: 16),
58                    Text(
59                        'Verificando cuenta...
```

```
70
71 // ✖ Si hay error o no existe el documento
72 if (userSnapshot.hasError ||
73     !userSnapshot.hasData ||
74     !userSnapshot.data!.exists) {
75     print('⚠ Error o documento no existe para usuario: ${user.uid}');
76     // Cerrar sesión y volver al login
77     WidgetsBinding.instance.addPostFrameCallback((_) async {
78         await FirebaseAuth.instance.signOut();
79         if (context.mounted) {
80             ScaffoldMessenger.of(context).showSnackBar(
81                 SnackBar(
82                     content: const Row(
83                         children: [
84                             Icon(Icons.error_outline, color: Colors.white),
85                             SizedBox(width: 12),
86                             Expanded(
87                                 child: Text(
118
119 // ✔ Verificar si la cuenta está activa
120 final bool isActive = userData['activo'] ?? true;
121
122 print('👤 Usuario: ${user.email}');
123 print('📄 Estado activo: $isActive');
124 print('👤 Rol: ${userData['rol']}');
125
126 // 🚫 Si la cuenta está DESACTIVADA, cerrar sesión
127 if (!isActive) {
128     print('✖ Cuenta DESACTIVADA - Cerrando sesión');
129     WidgetsBinding.instance.addPostFrameCallback((_) async {
130         await FirebaseAuth.instance.signOut();
131         if (context.mounted) {
132             ScaffoldMessenger.of(context).showSnackBar(
133                 SnackBar(
```

Esto ya se entiende por el nombre de la variable y por el flujo del código, entonces esos comentarios no son indispensables.

○ Fail Fast.

El código valida inmediatamente los casos críticos antes de permitir el acceso:

- si no hay usuario autenticado → devuelve la pantalla de login
- si el documento del usuario no existe → cierra sesión
- si los datos del usuario son nulos → vuelve al login
- si la cuenta está desactivada → cierra sesión y bloquea el acceso

```
27
28   @override
29   Widget build(BuildContext context) {
30     return StreamBuilder<User?>(
31       stream: FirebaseAuth.instance.authStateChanges(),
32       builder: (context, authSnapshot) {
33         // Si no hay usuario autenticado, mostrar login
34         if (!authSnapshot.hasData || authSnapshot.data == null) {
35           print('🔒 No hay usuario autenticado - Mostrando login');
36           return loginScreen;
37         }
38       }
39     );
40   }
41 }
```

```
70
71     // ❌ Si hay error o no existe el documento
72     if (userSnapshot.hasError ||
73         !userSnapshot.hasData ||
74         !userSnapshot.data!.exists) {
75       print('⚠️ Error o documento no existe para usuario: ${user.uid}');
76       // Cerrar sesión y volver al login
77       WidgetsBinding.instance.addPostFrameCallback((_) async {
78         await FirebaseAuth.instance.signOut();
79         return loginScreen;
80       });
81     }
82   }
83 }
```

```
106
107     // 📦 Obtener datos del usuario
108     final userData = userSnapshot.data!.data() as Map<String, dynamic>?;
109
110     if (userData == null) {
111       print('⚠️ userData es null para usuario: ${user.uid}');
112       // Si no hay datos, cerrar sesión
113       WidgetsBinding.instance.addPostFrameCallback((_) async {
114         await FirebaseAuth.instance.signOut();
115       });
116       return loginScreen;
117     }
118   }
119 }
```

```
// Si la cuenta está DESACTIVADA, cerrar sesión
if (!isActive) {
  print('✗ Cuenta DESACTIVADA - Cerrando sesión');
  WidgetsBinding.instance.addPostFrameCallback((_) async {
    await FirebaseAuth.instance.signOut();
    if (context.mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: const Row(
            children: [
              Icon(Icons.block, color: Colors.white, size: 28),
              SizedBox(width: 12),
              Expanded(
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.start,
                  mainAxisAlignment: MainAxisAlignment.min,
                  children: [
                    Text(
                      'Cuenta Desactivada',
                      style: TextStyle(
                        fontSize: 16,
                        fontWeight: FontWeight.bold,
                      ), // TextStyle
                    ), // Text
                    SizedBox(height: 4),
                    Text(
                      'Tu cuenta ha sido desactivada. Contacta al administrador para más información.',
                      style: TextStyle(fontSize: 14)
                    )
                  ],
                ),
              ),
            ],
          ),
        ),
      );
    }
  });
}
```

Eso es exactamente “Fail Fast”: detectar el problema lo más pronto posible y evitar continuar con un estado inválido.

○ Evita los números mágicos.

Hay valores fijos que aparecen sin explicación clara, por ejemplo:

- rutas como `"/login"`, `"/perfil"`, `"/inicio"`
- colores como `0xFF4D9EF6` y `0xFFFF5F6FA`
- duración de mensajes como `Duration(seconds: 4)` o `Duration(seconds: 6)`
- tamaños fijos como `16`, `28`, `12`, `10`, etc.

Eso son números mágicos.

La regla dice que:

- si un valor tiene un significado importante, debe estar nombrado con una constante
- no dejarlo “flotando” en el código

```
46  *const MyApp({super.key});
47
48  @override
49  Widget build(BuildContext context) {
50    final api = ApiDniServicio(bearerToken: bearer);
51
52    return MaterialApp(
53      debugShowCheckedModeBanner: false,
54      title: 'SOS Mascota',
55      navigatorKey: navigatorKey,
56      theme: ThemeData(
57        useMaterial3: true,
58        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF4D9EF6)),
59        scaffoldBackgroundColor: const Color(0xFFF5F6FA),
60        appBarTheme: const AppBarTheme(
61          backgroundColor: Colors.white,
62          foregroundColor: Colors.black,
63          elevation: 0.5,
64          centerTitle: true,
65          titleTextStyle: TextStyle(
66            fontSize: 18,
67            fontWeight: FontWeight.bold,
68            color: Colors.black,
69          ), // TextStyle
70
71      ),
72
73      // Content
74      home: const Row(
75        children: [
76          Icon(Icons.error_outline, color: Colors.white),
77          SizedBox(width: 12),
78          Expanded(
79            child: Text(
80              'Error al cargar los datos del usuario.',
81              style: TextStyle(fontSize: 15),
82            ), // Text
83          ), // Expanded
84        ],
85      ), // Row
86      backgroundColor: Colors.red,
87      duration: const Duration(seconds: 4),
88      behavior: SnackbarBehavior.floating,
89      shape: RoundedRectangleBorder(
90        borderRadius: BorderRadius.circular(10),
91      ), // RoundedRectangleBorder
92    ), // Snackbar
93  );
94
95  });
96
97  return loginScreen;
98
99  }
100
101  // Obtener datos del usuario
102  final userData = userSnapshot.data!.data() as Map<String, dynamic>;
103
104  if (userData == null) {
105    print('⚠️ userData es null para usuario: ${user.uid}');
```

“En la lógica de autenticación se detectan números mágicos en rutas, colores y tiempos de mensajes, lo que reduce la legibilidad del código y dificulta futuras modificaciones. Se recomienda reemplazarlos por constantes con nombres significativos.”

```
29 widget build(BuildContext context) {
112 // Si no hay datos, cerrar sesión
113 WidgetsBinding.instance.addPostFrameCallback((_) async {
114   await FirebaseAuth.instance.signOut();
115 });
116 return loginScreen;
117 }
118
119 // ✓ Verificar si la cuenta está activa
120 final bool isActive = userData['activo'] ?? true;
121
122 print('👤 Usuario: ${user.email}');
123 print('📌 Estado activo: $isActive');
124 print('👤 Rol: ${userData['rol']}');
125
126 // 🚫 Si la cuenta está DESACTIVADA, cerrar sesión
127 if (!isActive) {
128   print('❌ Cuenta DESACTIVADA - Cerrando sesión');
129   WidgetsBinding.instance.addPostFrameCallback((_) async {
130     await FirebaseAuth.instance.signOut();
131     if (context.mounted) {
132       ScaffoldMessenger.of(context).showSnackBar(
133         SnackBar(
134           content: const Row(
135             children: [
136               Icon(Icons.block, color: Colors.white, size: 28),
137               SizedBox(width: 12),
138               Expanded(
139                 child: Column(
140                   crossAxisAlignment: CrossAxisAlignment.start,
141                   mainAxisSize: MainAxisSize.min,
142                   children: [
143                     Text(
144                       'Cuenta Desactivada',
145                       style: TextStyle(
146                         fontSize: 16,
147                         fontWeight: FontWeight.bold,
148                       ), // TextStyle
149                     ), // Text
150                     SizedBox(height: 4),
151                     Text(
152                       'Tu cuenta ha sido desactivada. Contacta al administrador',
153                       style: TextStyle(fontSize: 14),
154                     ), // Text
155                   ],
156                 ), // Column
157               ), // Expanded
158             ],
159           ), // Row
160           backgroundColor: Colors.red.shade700,
161           duration: const Duration(seconds: 6),
162           behavior: SnackBarBehavior.floating,
163           shape: RoundedRectangleBorder(
164             borderRadius: BorderRadius.circular(12),
165           ), // RoundedRectangleBorder
166           margin: const EdgeInsets.all(16),
167         ), // SnackBar

```


Y en `auth_wrapper.dart` hay variables como:

- `authSnapshot`
- `userSnapshot`
- `userData`
- `isActive`

```
27
28  @override
29  Widget build(BuildContext context) {
30    return StreamBuilder<User?>(
31      stream: FirebaseAuth.instance.authStateChanges(),
32      builder: (context, authSnapshot) {
33        // Si no hay usuario autenticado, mostrar login
34        if (!authSnapshot.hasData || authSnapshot.data == null) {
35          print('🔒 No hay usuario autenticado - Mostrando login');
36          return loginScreen;
37        }
38
39        final user = authSnapshot.data!;
```

```
40      ),
41      snapshots(),
42      builder: (context, userSnapshot) {
43        // ⌚ Mientras carga, mostrar indicador
44        if (userSnapshot.connectionState == ConnectionState.waiting)
45          return const Scaffold(
46            backgroundColor: Colors.white,
47            body: Center(
48              child: Column(
49                mainAxisAlignment: MainAxisAlignment.center,
50                children: [
51                  CircularProgressIndicator(),
52                  SizedBox(height: 16),
53                  Text(
54                    'Verificando cuenta...',
55                    style: TextStyle(
56                      fontSize: 16,
57                      color: Colors.grey,
58                    ), // TextStyle
59                  ), // Text
60                ],
61              ), // Column
62            ), // Center
63          ); // Scaffold
64      },
65    );
66
67    // ❌ Si hay error o no existe el documento
68    if (userSnapshot.hasError ||
69        !userSnapshot.hasData ||
70        !userSnapshot.data!.exists) {
```

```
// 📦 Obtener datos del usuario
final userData = userSnapshot.data!.data() as Map<String, dynamic>;

if (userData == null) {
  print('⚠️ userData es null para usuario: ${user.uid}');
  // Si no hay datos, cerrar sesión
  WidgetsBinding.instance.addPostFrameCallback((_) async {
    await FirebaseAuth.instance.signOut();
  });
  return loginScreen;
}
```

```
// 🚫 Si la cuenta está DESACTIVADA, cerrar sesión
if (!isActive) {
  print('❌ Cuenta DESACTIVADA - Cerrando sesión');
  WidgetsBinding.instance.addPostFrameCallback((_) async {
    await FirebaseAuth.instance.signOut();
    if (context.mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: const Row(
```

- No uses variables globales.

En app.dart::

- final String bearer = "...";
- final GlobalKey<NavigatorState> navigatorKey = GlobalKey<NavigatorState>();

Eso sí es una variable global.

Por qué es un problema

- tiene un nombre poco significativo
- puede cambiar desde cualquier parte
- es accesible desde cualquier lugar del programa
- dificulta el mantenimiento y aumenta el riesgo

En este proyecto:

- bearer es un token JWT(token de autenticación para validar al usuario), y está declarado globalmente
- navigatorKey sí es útil, pero también está global

Eso se considera una mala práctica para la seguridad y la organización.

```
37
38 import 'servicios/api_dni_servicio.dart';
39
40
41 final String bearer =
42     "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyOTUsImV4cCI6MTc1ODIzOT
43 final GlobalKey<NavigatorState> navigatorKey = GlobalKey<NavigatorState>();
44
```

- Devuelve valores, no los imprimas.

En `auth_wrapper.dart`: hay varios `print()` y `debugPrint` dentro de la lógica de autenticación:

- `print('🔒 No hay usuario autenticado - Mostrando login');`
- `print('⚠️ Error o documento no existe para usuario: ${user.uid}');`
- `print('⚠️ userData es null para usuario: ${user.uid}');`
- `print('🔍 Usuario: ${user.email}');`
- `print('❌ Cuenta DESACTIVADA - Cerrando sesión');`
- `print('✅ Cuenta ACTIVA - Permitiendo acceso');`

En mi caso, el flujo de autenticación:

- decide si mostrar login
- decide si cerrar sesión
- decide si permitir acceso

Pero en lugar de devolver un estado claro, se están mandando mensajes por consola.

```
29 widget build(BuildContext context) {
30     return StreamBuilder<User?>(
31         stream: FirebaseAuth.instance.authStateChanges(),
32         builder: (context, authSnapshot) {
33             // ⛔ Si no hay usuario autenticado, mostrar login
34             if (!authSnapshot.hasData || authSnapshot.data == null) {
35                 print('🔒 No hay usuario autenticado - Mostrando login');
36                 return loginScreen;
37             }
38         }
39     );
40 }
```

```
73 ..... !userSnapshot.hasData ||  
74 ..... !userSnapshot.data!.exists) {  
75 .....   print(' ⚠ Error o documento no existe para usuario: ${user.uid}  
76 .....   // Cerrar sesión y volver al login  
77 .....   WidgetsBinding.instance.addPostFrameCallback((_) async {  
78 .....     await FirebaseAuth.instance.signOut();  
79 .....     if (context.mounted) {  
80 .....       ScaffoldMessenger.of(context).showSnackBar(  

```

```
109 .....  
110 .....   if (userData == null) {  
111 .....     print(' ⚠ userData es null para usuario: ${user.uid}');  
112 .....     // Si no hay datos, cerrar sesión  
113 .....     WidgetsBinding.instance.addPostFrameCallback((_) async {  
114 .....       await FirebaseAuth.instance.signOut();  
115 .....     });  
116 .....     return loginScreen;  
117 .....   }  

```

```
126 ..... // ⛔ Si la cuenta está DESACTIVADA, cerrar sesión  
127 ..... if (!isActive) {  
128 .....   print(' ✖ Cuenta DESACTIVADA - Cerrando sesión');  
129 .....   WidgetsBinding.instance.addPostFrameCallback((_) async {  
130 .....     await FirebaseAuth.instance.signOut();  
131 .....     if (context.mounted) {  
132 .....       ScaffoldMessenger.of(context).showSnackBar(  

```

```
118 .....  
119 ..... // ✅ Verificar si la cuenta está activa  
120 ..... final bool isActive = userData['activo'] ?? true;  
121 .....  
122 .....   print(' 🔍 Usuario: ${user.email}');  
123 .....   print(' 🚩 Estado activo: $isActive');  
124 .....   print(' 👤 Rol: ${userData['rol']}');
```

Donde se usan print() para informar el estado de la autenticación en lugar de devolver un resultado o manejarlo como valor.

- Utiliza los espacios en blanco para mejor lectura.

Se ve una buena utilización de espacios y sangría en auth_wrapper.dart.

Qué está bien

Hay:

- indentación correcta

- líneas separadas lógicamente
- bloques bien organizados
- espacios entre estructuras y dentro de widgets

Por ejemplo:

- `if (...) {`
- `return const Scaffold(...)`
- `children: [ ... ]`
- `content: const Row(...)`

Todo eso hace que el código sea más legible.

```
// SI no hay usuario autenticado, mostrar login
if (!authSnapshot.hasData || authSnapshot.data == null) {
  print('🔒 No hay usuario autenticado - Mostrando login');
  return loginScreen;
}

final user = authSnapshot.data!;

// ⏰ Escuchar cambios en el documento del usuario en tiempo real
return StreamBuilder<DocumentSnapshot>(
  stream: FirebaseFirestore.instance
    .collection('usuarios')
    .doc(user.uid)
    .snapshots(),
  builder: (context, userSnapshot) {
    // ⏰ Mientras carga, mostrar indicador
    if (userSnapshot.connectionState == ConnectionState.waiting) {
      return const Scaffold(
        backgroundColor: Colors.white,
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              CircularProgressIndicator(),
            ],
          ),
        ),
      );
    }
    // ⏰ Mientras carga, mostrar indicador
    if (userSnapshot.connectionState == ConnectionState.waiting) {
      return const Scaffold(
        backgroundColor: Colors.white,
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              CircularProgressIndicator(),
            ],
          ),
        ),
      );
    }
  },
);
```

```
130         await FirebaseAuth.instance.signOut(),
131       if (context.mounted) {
132         ScaffoldMessenger.of(context).showSnackBar(
133           SnackBar(
134             content: const Row(
135               children: [
136                 Icon(Icons.block, color: Colors.white, size: 24),
137                 SizedBox(width: 12),
138                 Expanded(
139                   child: Column(
140                     crossAxisAlignment: CrossAxisAlignment.start,
141                     mainAxisAlignment: MainAxisAlignment.min,
142                     children: [
143                       Text(
144                         'Cuenta Desactivada',
145                         style: TextStyle(
146                           fontSize: 16,
147                           fontWeight: FontWeight.bold,
148                         ), // TextStyle
149                       ), // Text
150                       SizedBox(height: 4),
151                       Text(
152                         'Tu cuenta ha sido desactivada. Contacta',
153                         style: TextStyle(fontSize: 14),
154                       ), // Text
155                     ],
156                   ),
157                 ),
158             ),
159           ),
160         ),
161       ),
162     ),
163   ),
164 ),
165 ),
166 ),
167 ),
168 ),
169 ),
170 ),
171 ),
172 ),
173 ),
174 ),
175 ),
176 ),
177 ),
178 ),
179 ),
180 ),
181 ),
182 ),
183 ),
184 ),
185 ),
186 ),
187 ),
188 ),
189 ),
190 ),
191 ),
192 ),
193 ),
194 ),
195 ),
196 ),
197 ),
198 ),
199 ),
200 ),
201 ),
202 ),
203 ),
204 ),
205 ),
206 ),
207 ),
208 ),
209 ),
210 ),
211 ),
212 ),
213 ),
214 ),
215 ),
216 ),
217 ),
218 ),
219 ),
220 ),
221 ),
222 ),
223 ),
224 ),
225 ),
226 ),
227 ),
228 ),
229 ),
230 ),
231 ),
232 ),
233 ),
234 ),
235 ),
236 ),
237 ),
238 ),
239 ),
240 ),
241 ),
242 ),
243 ),
244 ),
245 ),
246 ),
247 ),
248 ),
249 ),
250 ),
251 ),
252 ),
253 ),
254 ),
255 ),
256 ),
257 ),
258 ),
259 ),
260 ),
261 ),
262 ),
263 ),
264 ),
265 ),
266 ),
267 ),
268 ),
269 ),
270 ),
271 ),
272 ),
273 ),
274 ),
275 ),
276 ),
277 ),
278 ),
279 ),
280 ),
281 ),
282 ),
283 ),
284 ),
285 ),
286 ),
287 ),
288 ),
289 ),
290 ),
291 ),
292 ),
293 ),
294 ),
295 ),
296 ),
297 ),
298 ),
299 ),
300 ),
301 ),
302 ),
303 ),
304 ),
305 ),
306 ),
307 ),
308 ),
309 ),
310 ),
311 ),
312 ),
313 ),
314 ),
315 ),
316 ),
317 ),
318 ),
319 ),
320 ),
321 ),
322 ),
323 ),
324 ),
325 ),
326 ),
327 ),
328 ),
329 ),
330 ),
331 ),
332 ),
333 ),
334 ),
335 ),
336 ),
337 ),
338 ),
339 ),
340 ),
341 ),
342 ),
343 ),
344 ),
345 ),
346 ),
347 ),
348 ),
349 ),
350 ),
351 ),
352 ),
353 ),
354 ),
355 ),
356 ),
357 ),
358 ),
359 ),
360 ),
361 ),
362 ),
363 ),
364 ),
365 ),
366 ),
367 ),
368 ),
369 ),
370 ),
371 ),
372 ),
373 ),
374 ),
375 ),
376 ),
377 ),
378 ),
379 ),
380 ),
381 ),
382 ),
383 ),
384 ),
385 ),
386 ),
387 ),
388 ),
389 ),
390 ),
391 ),
392 ),
393 ),
394 ),
395 ),
396 ),
397 ),
398 ),
399 ),
400 ),
401 ),
402 ),
403 ),
404 ),
405 ),
406 ),
407 ),
408 ),
409 ),
410 ),
411 ),
412 ),
413 ),
414 ),
415 ),
416 ),
417 ),
418 ),
419 ),
420 ),
421 ),
422 ),
423 ),
424 ),
425 ),
426 ),
427 ),
428 ),
429 ),
430 ),
431 ),
432 ),
433 ),
434 ),
435 ),
436 ),
437 ),
438 ),
439 ),
440 ),
441 ),
442 ),
443 ),
444 ),
445 ),
446 ),
447 ),
448 ),
449 ),
450 ),
451 ),
452 ),
453 ),
454 ),
455 ),
456 ),
457 ),
458 ),
459 ),
460 ),
461 ),
462 ),
463 ),
464 ),
465 ),
466 ),
467 ),
468 ),
469 ),
470 ),
471 ),
472 ),
473 ),
474 ),
475 ),
476 ),
477 ),
478 ),
479 ),
480 ),
481 ),
482 ),
483 ),
484 ),
485 ),
486 ),
487 ),
488 ),
489 ),
490 ),
491 ),
492 ),
493 ),
494 ),
495 ),
496 ),
497 ),
498 ),
499 ),
500 ),
501 ),
502 ),
503 ),
504 ),
505 ),
506 ),
507 ),
508 ),
509 ),
510 ),
511 ),
512 ),
513 ),
514 ),
515 ),
516 ),
517 ),
518 ),
519 ),
520 ),
521 ),
522 ),
523 ),
524 ),
525 ),
526 ),
527 ),
528 ),
529 ),
530 ),
531 ),
532 ),
533 ),
534 ),
535 ),
536 ),
537 ),
538 ),
539 ),
540 ),
541 ),
542 ),
543 ),
544 ),
545 ),
546 ),
547 ),
548 ),
549 ),
550 ),
551 ),
552 ),
553 ),
554 ),
555 ),
556 ),
557 ),
558 ),
559 ),
560 ),
561 ),
562 ),
563 ),
564 ),
565 ),
566 ),
567 ),
568 ),
569 ),
570 ),
571 ),
572 ),
573 ),
574 ),
575 ),
576 ),
577 ),
578 ),
579 ),
580 ),
581 ),
582 ),
583 ),
584 ),
585 ),
586 ),
587 ),
588 ),
589 ),
590 ),
591 ),
592 ),
593 ),
594 ),
595 ),
596 ),
597 ),
598 ),
599 ),
600 ),
601 ),
602 ),
603 ),
604 ),
605 ),
606 ),
607 ),
608 ),
609 ),
610 ),
611 ),
612 ),
613 ),
614 ),
615 ),
616 ),
617 ),
618 ),
619 ),
620 ),
621 ),
622 ),
623 ),
624 ),
625 ),
626 ),
627 ),
628 ),
629 ),
630 ),
631 ),
632 ),
633 ),
634 ),
635 ),
636 ),
637 ),
638 ),
639 ),
640 ),
641 ),
642 ),
643 ),
644 ),
645 ),
646 ),
647 ),
648 ),
649 ),
650 ),
651 ),
652 ),
653 ),
654 ),
655 ),
656 ),
657 ),
658 ),
659 ),
660 ),
661 ),
662 ),
663 ),
664 ),
665 ),
666 ),
667 ),
668 ),
669 ),
670 ),
671 ),
672 ),
673 ),
674 ),
675 ),
676 ),
677 ),
678 ),
679 ),
680 ),
681 ),
682 ),
683 ),
684 ),
685 ),
686 ),
687 ),
688 ),
689 ),
690 ),
691 ),
692 ),
693 ),
694 ),
695 ),
696 ),
697 ),
698 ),
699 ),
700 ),
701 ),
702 ),
703 ),
704 ),
705 ),
706 ),
707 ),
708 ),
709 ),
710 ),
711 ),
712 ),
713 ),
714 ),
715 ),
716 ),
717 ),
718 ),
719 ),
720 ),
721 ),
722 ),
723 ),
724 ),
725 ),
726 ),
727 ),
728 ),
729 ),
730 ),
731 ),
732 ),
733 ),
734 ),
735 ),
736 ),
737 ),
738 ),
739 ),
740 ),
741 ),
742 ),
743 ),
744 ),
745 ),
746 ),
747 ),
748 ),
749 ),
750 ),
751 ),
752 ),
753 ),
754 ),
755 ),
756 ),
757 ),
758 ),
759 ),
760 ),
761 ),
762 ),
763 ),
764 ),
765 ),
766 ),
767 ),
768 ),
769 ),
770 ),
771 ),
772 ),
773 ),
774 ),
775 ),
776 ),
777 ),
778 ),
779 ),
780 ),
781 ),
782 ),
783 ),
784 ),
785 ),
786 ),
787 ),
788 ),
789 ),
790 ),
791 ),
792 ),
793 ),
794 ),
795 ),
796 ),
797 ),
798 ),
799 ),
800 ),
801 ),
802 ),
803 ),
804 ),
805 ),
806 ),
807 ),
808 ),
809 ),
810 ),
811 ),
812 ),
813 ),
814 ),
815 ),
816 ),
817 ),
818 ),
819 ),
820 ),
821 ),
822 ),
823 ),
824 ),
825 ),
826 ),
827 ),
828 ),
829 ),
830 ),
831 ),
832 ),
833 ),
834 ),
835 ),
836 ),
837 ),
838 ),
839 ),
840 ),
841 ),
842 ),
843 ),
844 ),
845 ),
846 ),
847 ),
848 ),
849 ),
850 ),
851 ),
852 ),
853 ),
854 ),
855 ),
856 ),
857 ),
858 ),
859 ),
860 ),
861 ),
862 ),
863 ),
864 ),
865 ),
866 ),
867 ),
868 ),
869 ),
870 ),
871 ),
872 ),
873 ),
874 ),
875 ),
876 ),
877 ),
878 ),
879 ),
880 ),
881 ),
882 ),
883 ),
884 ),
885 ),
886 ),
887 ),
888 ),
889 ),
890 ),
891 ),
892 ),
893 ),
894 ),
895 ),
896 ),
897 ),
898 ),
899 ),
900 ),
901 ),
902 ),
903 ),
904 ),
905 ),
906 ),
907 ),
908 ),
909 ),
910 ),
911 ),
912 ),
913 ),
914 ),
915 ),
916 ),
917 ),
918 ),
919 ),
920 ),
921 ),
922 ),
923 ),
924 ),
925 ),
926 ),
927 ),
928 ),
929 ),
930 ),
931 ),
932 ),
933 ),
934 ),
935 ),
936 ),
937 ),
938 ),
939 ),
940 ),
941 ),
942 ),
943 ),
944 ),
945 ),
946 ),
947 ),
948 ),
949 ),
950 ),
951 ),
952 ),
953 ),
954 ),
955 ),
956 ),
957 ),
958 ),
959 ),
960 ),
961 ),
962 ),
963 ),
964 ),
965 ),
966 ),
967 ),
968 ),
969 ),
970 ),
971 ),
972 ),
973 ),
974 ),
975 ),
976 ),
977 ),
978 ),
979 ),
980 ),
981 ),
982 ),
983 ),
984 ),
985 ),
986 ),
987 ),
988 ),
989 ),
990 ),
991 ),
992 ),
993 ),
994 ),
995 ),
996 ),
997 ),
998 ),
999 ),
1000 ),
```

También en app.dart hay muchas rutas seguidas sin una separación más clara de módulos o grupos de navegación.

```
initialRoute: "/login",
routes: {
  "/login": (_) => ChangeNotifierProvider(
    create: (_) => LoginVM(),
    child: const PantallaLogin(),
  ), // ChangeNotifierProvider
  "/registro": (_) => ChangeNotifierProvider(
    create: (_) => RegistroVM(),
    //create: (_) => RegistroVM(apiDni: api),
    child: const PantallaRegistro(),
  ), // ChangeNotifierProvider
  "/registroAdmin": (_) => ChangeNotifierProvider(
    create: (_) => AdminVM(apiDni: api),
    child: const PantallaRegistroAdmin(),
  ), // ChangeNotifierProvider
  "/recuperar": (_) => ChangeNotifierProvider(
    create: (_) => RecuperarVM(),
    child: const PantallaRecuperar(),
  ), // ChangeNotifierProvider
  "/verificaEmail": (_) => const PantallaVerificaEmail(),
  "/perfil": (_) => ChangeNotifierProvider(
    create: (_) => PerfilVM(),
    child: const PantallaPerfil(),
  ), // ChangeNotifierProvider
  "/inicio": (_) => AuthWrapper(
    loginScreen: ChangeNotifierProvider(
      create: (_) => LoginVM(),
      child: const PantallaLogin(),
    ), // ChangeNotifierProvider
    homeScreenBuilder: (uid, userData) => ChangeNotifierProvider(
      create: (_) => NotificacionVM()..escucharNotificaciones(),
      child: const PantallaInicio(),
    ), // ChangeNotifierProvider
  ), // AuthWrapper
  "/inicioAdmin": (_) => const PantallaInicioAdmin(),
  "/adminUsuarios": (_) => const PantallaAdminUsuarios(),
  "/adminReportes": (_) => const PantallaAdminReporte(),
  "/adminNotificaciones": (_) => ChangeNotifierProvider(
    create: (_) => AdminNotificacionesVM(),
    child: const PantallaAdminNotificaciones(),
  ), // ChangeNotifierProvider
}
```

Código mejorado:

En app.dart:

No te repitas

Se centralizó la configuración con AppConfig y las rutas con AppRoutes, evitando repetir el mismo patrón de providers y pantallas en cada ruta.

Comenta lo necesario

Se dejó solo la documentación útil y clara para la configuración y la validación del flujo.

Fail Fast

La app ahora organiza mejor el arranque y la navegación desde una estructura central, reduciendo el riesgo de cargar rutas con configuración inválida.

Evita los números mágicos

Se sacaron valores como colores, tiempos y tamaños a app_config.dart, evitando que queden dispersos por el código.

Utiliza buenos nombres

Se usaron nombres descriptivos como:

- AppConfig
- AppRoutes
- login
- registro
- inicioAdmin

No uses variables globales

Se movió la configuración global a una clase centralizada, evitando que esos valores estén repartidos por todo el archivo.

Devuelve valores, no los imprimas

La lógica principal ya no depende de print() para decidir el flujo; se usa validación directa y retorno de pantallas.

Utiliza los espacios en blanco para mejor lectura

La separación entre configuración, rutas y widget principal mejora mucho la legibilidad del archivo.

```
40  /// Nombres de ruta centralizados como constantes.
41  /// Evita escribir el mismo string ("/login", "/perfil", etc.) suelto
42  /// en distintas partes del proyecto (strings "mágicos").
43  class AppRoutes {
44      static const String login = '/login';
45      static const String registro = '/registro';
46      static const String registroAdmin = '/registroAdmin';
47      static const String recuperar = '/recuperar';
48      static const String verificaEmail = '/verificaEmail';
49      static const String perfil = '/perfil';
50      static const String inicio = '/inicio';
51      static const String inicioAdmin = '/inicioAdmin';
52      static const String adminUsuarios = '/adminUsuarios';
53      static const String adminReportes = '/adminReportes';
54      static const String adminNotificaciones = '/adminNotificaciones';
55      static const String reportarMascota = '/reportarMascota';
56      static const String avistamiento = '/avistamiento';
57      static const String verReportes = '/verReportes';
58      static const String misReportes = '/misReportes';
59      static const String notificaciones = '/notificaciones';
60      static const String comentarios = '/comentarios';
61      static const String mapa = '/mapa';
62
63      static Map<String, WidgetBuilder> build() {
64          final api = ApiDniServicio(bearerToken: AppConfig.bearerToken);
65
66          return {
67              login: (_) => _withProvider(LoginVM.new, const PantallaLogin()),
68              registro: (_) => _withProvider(RegistroVM.new, const PantallaRegistro()),
69              registroAdmin: (_) =>
```

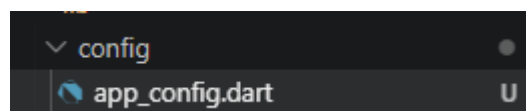
```
69         registroAdmin: (_) =>
70         | _withProvider(() => AdminVM(apiDni: api), const PantallaRegistroAdmin()),
71         recuperar: (_) => _withProvider(RecuperarVM.new, const PantallaRecuperar()),
72         verificaEmail: (_) => const PantallaVerificaEmail(),
73         perfil: (_) => _withProvider(PerfilVM.new, const PantallaPerfil()),
74
75         inicio: (_) => AuthWrapper(
76         | loginScreen: _withProvider(LoginVM.new, const PantallaLogin()),
77         | homeScreenBuilder: (uid, userData) =>
78         | | _withNotificaciones(const PantallaInicio()),
79         | ), // AuthWrapper
80
81         inicioAdmin: (_) => const PantallaInicioAdmin(),
82         adminUsuarios: (_) => const PantallaAdminUsuarios(),
83         adminReportes: (_) => const PantallaAdminReporte(),
84         adminNotificaciones: (_) => _withProvider(
85         | AdminNotificacionesVM.new,
86         | const PantallaAdminNotificaciones(),
87         | ),
88
89         reportarMascota: (_) => const PantallaReporteMascota(),
90         avistamiento: (_) => const PantallaAvistamiento(),
91         verReportes: (_) => const PantallaVerReportes(),
92         misReportes: (_) => const PantallaMisReportes(),
93
94         notificaciones: (_) => _withNotificaciones(const PantallaNotificaciones()),
95         comentarios: (_) => const PantallaComentarios(),
96         mapa: (_) => const PantallaMapaInteractivo(),
97     };
```

```
99
100     /// Envuelve [child] con el `ChangeNotifierProvider` de [create].
101     /// Reemplaza el bloque `ChangeNotifierProvider(create: ..., child: ...)`
102     /// que antes se repetía manualmente para cada pantalla con ViewModel.
103     static Widget _withProvider<T extends ChangeNotifier>(
104     | T Function() create,
105     | Widget child,
106     ) {
107     | return ChangeNotifierProvider<T>(create: (_) => create(), child: child);
108     | }
109
110     /// `NotificacionVM` se inicializaba igual en "/inicio" y "/notificaciones".
111     /// Se centraliza aquí para no repetir la misma lógica dos veces.
112     static Widget _withNotificaciones(Widget child) {
113     | return ChangeNotifierProvider(
114     | | create: (_) => NotificacionVM()..escucharNotificaciones(),
115     | | child: child,
116     | ); // ChangeNotifierProvider
117     | }
118 }
```

```
120 class MyApp extends StatelessWidget {
121   const MyApp({super.key});
122
123   @override
124   Widget build(BuildContext context) {
125     return MaterialApp(
126       debugShowCheckedModeBanner: false,
127       title: AppConfig.appTitle,
128       theme: ThemeData(
129         useMaterial3: true,
130         colorScheme: ColorScheme.fromSeed(seedColor: ■ AppConfig.primaryColor),
131         scaffoldBackgroundColor: ■ AppConfig.scaffoldBackground,
132         appBarTheme: const AppBarTheme(
133           backgroundColor: ■ Colors.white,
134           foregroundColor: □ Colors.black,
135           elevation: 0.5,
136           centerTitle: true,
137           titleTextStyle: TextStyle(
138             fontSize: 18,
139             fontWeight: FontWeight.bold,
140             color: □ Colors.black,
141           ), // TextStyle
142         ), // AppBarTheme
143       ), // ThemeData
144       initialRoute: AppRoutes.login,
145       routes: AppRoutes.build(),
146     ); // MaterialApp
147   }
148 }
```

Se creó una ruta en config/app_config.dart.

```
5 import 'package:sos_mascotas/vista/usuario/pantalla_notificacion.dart';
6 import 'package:sos_mascotas/vistamodelo/notificacion/notificacion_vm.dart';
7 import 'config/app_config.dart';
8 import 'utils/auth_wrapper.dart';
9
```



```
1  /// Configuración global de la app
2  ///
3  /// Centraliza valores que antes estaban repetidos como "números mágicos"
4  /// (colores, duraciones, tamaños de SnackBar, etc.) para que un cambio
5  /// de diseño no obligue a tocar varios archivos. `auth_wrapper.dart`
6  /// importa esta clase desde aquí (import circular permitido en Dart).
7  import 'package:flutter/material.dart';
8
9  class AppConfig {
10   AppConfig._(); // Evita instanciar esta clase; solo se usa de forma estática.
11
12   static const String appTitle = 'SOS Mascota';
13
14   static const String bearerToken = String.fromEnvironment(
15     'BEARER_TOKEN',
16     defaultValue: '',
17   );
18
19   // Colores base del tema
20   static const Color primaryColor = Color(0xFF4D9EF6);
21   static const Color scaffoldBackground = Color(0xFFF5F6FA);
22   // Duraciones de SnackBar
23   static const Duration snackDurationDefault = Duration(seconds: 4);
24   static const Duration snackDurationLong = Duration(seconds: 6);
25   // Tamaños y espaciados usados en los avisos (SnackBars) de AuthWrapper
26   static const double snackIconSize = 28;
27   static const double snackSpacing = 12;
28   static const double snackTextSpacing = 4;
29   static const double snackBorderRadiusDefault = 10;
30   static const double snackBorderRadiusLarge = 12;
31   static const double snackMargin = 16;
32 }
```

En auth_wrapper.dart.

No te repitas

Se unificó la lógica de cierre de sesión y mensaje de error para evitar duplicar el mismo proceso en varios casos.

Comenta lo necesario

Se dejó solo la documentación clave sobre lo que hace AuthWrapper y cómo valida la sesión.

Fail Fast

Se valida inmediatamente:

- si no hay sesión
- si el documento del usuario no existe
- si el usuario es nulo
- si la cuenta está desactivada

Si algo falla, devuelve al login de inmediato.

Evita los números mágicos

Se usaron constantes de `app_config.dart` para:

- tiempo del `SnackBar`
- tamaños
- radios
- espaciados

Utiliza buenos nombres

Los nombres `AuthWrapper`, `homeScreenBuilder`, `cuentaActiva` y `docExiste` son claros y descriptivos.

No uses variables globales

La validación se hace local a la sesión y al documento, sin depender de valores globales a lo largo de la app.

Devuelve valores, no los imprimas

Se eliminó la lógica de depuración por consola y se reemplazó por decisiones de flujo y mensajes visibles para el usuario.

Utiliza los espacios en blanco para mejor lectura

La estructura del `StreamBuilder`, validaciones y retorno está bien separada en bloques.

```
1 import 'package:flutter/material.dart';
2 import 'package:firebase_auth/firebase_auth.dart';
3 import 'package:cloud_firestore/cloud_firestore.dart';
4
5 import '../config/app_config.dart';
6
7 /// Verifica en tiempo real que el usuario esté autenticado, que su
8 /// documento exista en Firestore y que su cuenta siga activa.
9 ///
10 /// Aplica "fail fast": ante cualquier condición inválida (sin sesión,
11 /// documento inexistente, error de lectura o cuenta desactivada) cierra
12 /// sesión de inmediato y regresa a `loginScreen`, en vez de dejar que
13 /// la app siga en un estado a medias.
14 class AuthWrapper extends StatelessWidget {
15   final Widget loginScreen;
16   final Widget Function(String uid, Map<String, dynamic> userData) homeScreenBuilder;
17
18   const AuthWrapper({
19     super.key,
20     required this.loginScreen,
21     required this.homeScreenBuilder,
22   });
23
24   @override
25   Widget build(BuildContext context) {
26     return StreamBuilder<User?>(
27       stream: FirebaseAuth.instance.authStateChanges(),
28       builder: (context, authSnapshot) {
29         final user = authSnapshot.data;
```

```
24   @override
25   Widget build(BuildContext context) {
26     return StreamBuilder<User?>(
27       stream: FirebaseAuth.instance.authStateChanges(),
28       builder: (context, authSnapshot) {
29         final user = authSnapshot.data;
30         if (user == null) return loginScreen; // Fail fast: sin sesión, no hay nada más que revisar
31
32         return StreamBuilder<DocumentSnapshot>(
33           stream: FirebaseFirestore.instance
34             .collection('usuarios')
35             .doc(user.uid)
36             .snapshots(),
37           builder: (context, userSnapshot) {
38             if (userSnapshot.connectionState == ConnectionState.waiting) {
39               return const _VerificandoCuenta();
40             }
41
42             final docExiste = userSnapshot.hasData && userSnapshot.data!.exists;
43             if (userSnapshot.hasError || !docExiste) {
44               _cerrarSesionYAvisar(
45                 context,
46                 mensaje: 'Error al cargar los datos del usuario.',
47                 icono: Icons.error_outline,
48                 color: Colors.red,
49                 duracion: AppConfig.snackDurationDefault,
50               );
51               return loginScreen;
52             }
53           }
54         );
55       }
56     );
57   }
```

```
53
54     final userData = userSnapshot.data!.data() as Map<String, dynamic>;
55     if (userData == null) {
56         FirebaseAuth.instance.signOut();
57         return loginScreen;
58     }
59
60     final bool cuentaActiva = userData['activo'] ?? true;
61     if (!cuentaActiva) {
62         _cerrarSesionYAvisar(
63             context,
64             titulo: 'Cuenta Desactivada',
65             mensaje:
66                 'Tu cuenta ha sido desactivada. Contacta al administrador para más información.',
67             icono: Icons.block,
68             color: Colors.red.shade700,
69             duracion: AppConfig.snackDurationLong,
70         );
71         return loginScreen;
72     }
73
74     return homeScreenBuilder(user.uid, userData);
75 },
76 ); // StreamBuilder
77 },
78 ); // StreamBuilder
79 }
```

```
80
81 /// Cierra sesión y muestra un SnackBar con el motivo.
82 /// Antes este mismo patrón (signOut + SnackBar) estaba duplicado
83 /// para el caso de "error al cargar datos" y el de "cuenta desactivada".
84 void _cerrarSesionYAvisar(
85     BuildContext context, {
86     String? titulo,
87     required String mensaje,
88     required IconData icono,
89     required Color color,
90     required Duration duracion,
91 }) {
92     WidgetsBinding.instance.addPostFrameCallback((_) async {
93         await FirebaseAuth.instance.signOut();
94         if (!context.mounted) return;
95
96         final esAvisoImportante = titulo != null;
97
98         ScaffoldMessenger.of(context).showSnackBar(
99             SnackBar(
100                 content: Row(
101                     children: [
102                         Icon(icono, color: Colors.white, size: AppConfig.snackIconSize),
103                         SizedBox(width: AppConfig.snackSpacing),
104                         Expanded(
105                             child: Column(
106                                 crossAxisAlignment: CrossAxisAlignment.start,
107                                 mainAxisAlignment: MainAxisAlignment.min,
108                                 children: [
109                                     if (esAvisoImportante) ...[
110                                         Text(
```

```
131         ), // RoundedRectangleBorder
132         margin: esAvisoImportante ? const EdgeInsets.all(AppConfig.snackMargin) : null,
133       ), // SnackBar
134     );
135   });
136 }
137 }
138
139 /// Pantalla de carga mientras se valida el estado de la cuenta contra Firestore.
140 class _VerificandoCuenta extends StatelessWidget {
141   const _VerificandoCuenta();
142
143   @override
144   Widget build(BuildContext context) {
145     return const Scaffold(
146       backgroundColor: Colors.white,
147       body: Center(
148         child: Column(
149           mainAxisAlignment: MainAxisAlignment.center,
150           children: [
151             CircularProgressIndicator(),
152             SizedBox(height: 16),
153             Text('Verificando cuenta...', style: TextStyle(fontSize: 16, color: Colors.grey)),
154           ],
155         ), // Column
156       ), // Center
157     ); // Scaffold
158   }
159 }
```

Se corrigió la autenticación de usuario validando la sesión en Firebase Auth, la existencia del documento del usuario en Firestore y el estado activo de la cuenta. Además, se redirige al login cuando la cuenta no cumple con los requisitos de acceso. En app.dart se reorganizó la navegación y la configuración para reducir la repetición y mejorar la mantenibilidad.

Autenticar Usuario se realizó en estos 2 archivos por:

- auth_wrapper.dart: validar sesión, validar identidad, permitir o bloquear acceso, controlar usuarios activos/inactivos.
- app.dart : ayuda a organizar la navegación y configuración.

- Registrar reporte de mascota

En pantalla_reporte_mascota.dart

- No te repitas.

El bloque completo de "seleccionar imagen → mostrar snackbar de análisis → subir foto → mostrar snackbar de éxito/error" está copiado 3 veces casi idéntico: en el onTap de la imagen principal (ImageSource.gallery)

```
223 onTap: () async {
224   final picker = ImagePicker();
225   final picked = await picker.pickImage(
226     source: ImageSource.gallery,
227   );
228
229   if (picked != null) {
230     try {
231       ScaffoldMessenger.of(context).showSnackBar(
232         const SnackBar(
233           content: Text("Analizando imagen... 🟡"),
234           backgroundColor: Colors.blueAccent,
235           duration: Duration(seconds: 2),
236         ), // SnackBar
237       );
238
239       dynamic imagenParaEnviar;
240
241       if (kIsWeb) {
242         // En Web -> usamos bytes
243         imagenParaEnviar = await picked.readAsBytes();
244       } else {
245         // En Android/iOS -> usamos File normal
246         imagenParaEnviar = File(picked.path);
247       }
248
249       final url = await vm.subirFoto(imagenParaEnviar);
250       vm.agregarFoto(url);
251     }
```

```
252 ScaffoldMessenger.of(context).showSnackBar(  
253   const SnackBar(  
254     content: Text(  
255       "✅ Imagen válida detectada y subida correctamente",  
256     ), // Text  
257     backgroundColor: Colors.green,  
258   ), // SnackBar  
259 );  
260 } catch (e) {  
261   ScaffoldMessenger.of(context).showSnackBar(  
262     SnackBar(  
263       content: Text(  
264         e.toString().replaceAll("Exception: ", ""),  
265         style: const TextStyle(  
266           fontWeight: FontWeight.bold,  
267         ), // TextStyle  
268       ), // Text  
269       backgroundColor: Colors.redAccent,  
270     ), // SnackBar  
271   );  
272 }  
273 }  
274 },
```

En el botón "Cámara" (ImageSource.camera)

```
386 // ... (onPressed y estilo)  
387 onPressed: () async {  
388   final picker = ImagePicker();  
389   final picked = await picker.pickImage(  
390     source: ImageSource.camera,  
391   );  
392   if (picked == null) return;  
393  
394   try {  
395     ScaffoldMessenger.of(context).showSnackBar(  
396       const SnackBar(  
397         content: Text("Analizando imagen... 🧠"),  
398         backgroundColor: Colors.blueAccent,  
399         duration: Duration(seconds: 2),  
400       ), // SnackBar  
401     );  
402  
403     dynamic imagenParaVM;  
404  
405     if (kIsWeb) {  
406       // En Web -> usamos bytes del archivo  
407       imagenParaVM = await picked.readAsBytes();  
408     } else {  
409       // En Android/iOS -> usamos File normal  
410       imagenParaVM = File(picked.path);  
411     }  
412  
413     final url = await vm.subirFoto(imagenParaVM);  
414   }
```

```
415 vm.agregarFoto(url);
416
417 ScaffoldMessenger.of(context).showSnackBar(
418   const SnackBar(
419     content: Text(
420       "✅ Imagen válida detectada y subida correctamente",
421     ), // Text
422     backgroundColor: Colors.green,
423   ), // SnackBar
424 );
425 } catch (e) {
426   ScaffoldMessenger.of(context).showSnackBar(
427     SnackBar(
428       content: Text(
429         e.toString().replaceAll("Exception: ", ""),
430         style: const TextStyle(
431           fontWeight: FontWeight.bold,
432         ), // TextStyle
433       ), // Text
434       backgroundColor: Colors.redAccent,
435     ), // SnackBar
436   );
437 }
438 },
```

En el botón "Galería" (ImageSource.gallery otra vez)

```
457 // ... (onPressed y estilo)
458 onPressed: () async {
459   final picker = ImagePicker();
460   final picked = await picker.pickImage(
461     source: ImageSource.gallery,
462   );
463   if (picked == null) return;
464
465   try {
466     ScaffoldMessenger.of(context).showSnackBar(
467       const SnackBar(
468         content: Text("Analizando imagen... 🌀"),
469         backgroundColor: Colors.blueAccent,
470         duration: Duration(seconds: 2),
471       ), // SnackBar
472     );
473
474     dynamic imagenParaVM;
475
476     if (kIsWeb) {
477       // 🌐 WEB: el picker devuelve bytes, NO File
478       imagenParaVM = await picked.readAsBytes();
479     } else {
480       // 📁 MOVIL: el picker devuelve un path válido
481       imagenParaVM = File(picked.path);
482     }
483
484     final url = await vm.subirFoto(imagenParaVM);
485     vm.agregarFoto(url);
```

```
487 ScaffoldMessenger.of(context).showSnackBar(  
488   const SnackBar(  
489     content: Text(  
490       "✅ Imagen válida detectada y subida correctamente",  
491     ), // Text  
492     backgroundColor: Colors.green,  
493   ), // SnackBar  
494 );  
495 } catch (e) {  
496   ScaffoldMessenger.of(context).showSnackBar(  
497     SnackBar(  
498       content: Text(  
499         e.toString().replaceAll("Exception: ", ""),  
500       style: const TextStyle(  
501         fontWeight: FontWeight.bold,  
502       ), // TextStyle  
503     ), // Text  
504     backgroundColor: Colors.redAccent,  
505   ), // SnackBar  
506 );  
507 }  
508 },
```

Los tres hacen exactamente lo mismo (seleccionar imagen → SnackBar "Analizando imagen..." → subir → SnackBar de éxito/error), solo cambia el ImageSource.

También se repite la lógica de "S/." al mostrar el monto (la función anónima (() { ... })()):

Si el monto se formatea distinto en más de un lugar, terminará duplicándose también.

```
943 if (r.montoRecompensa.isNotEmpty) ...[  
944   (() {  
945     String m = r.montoRecompensa.trim();  
946     if (!m.startsWith(RegExp(r'^S/\.?s*')) {  
947       m = 'S/. $m';  
948     }  
949     return _infoRow("💰 Monto", m);  
950   })
```

- Comenta lo necesario.

Hay comentarios que sí ayudan (ej. explicar por qué se usa .skip(1) para las miniaturas). Pero hay varios que sobran o son señales de código incompleto:

- Definiciones auxiliares (_sectionTitle, _card, etc.) no incluidas aquí por espacio, pero son necesarias para que el código compile. esto es una nota de "borrador", no debería quedar en el código final.

```

95     ) async {
96         // Definiciones auxiliares (_sectionTitle, _card, etc.) no incluidas aquí por espacio,
97         // pero son necesarias para que el código compile.
98
99         return showDialog(
100             context: context,
101             builder: (BuildContext context) {
102                 return AlertDialog(
103                     title: const Text("Eliminar foto"),

```

- // ... (onPressed y estilo) y // ... (Icono de cámara y textos) : comentarios placeholder que no aportan nada y quedaron de cuando se recortó el código.

```

386         // ... (onPressed y estilo)
387         onPressed: () async {
388             final picker = ImagePicker();
389             final picked = await picker.pickImage(
390                 source: ImageSource.camera,
391             );
392             if (picked == null) return;
393

```

```

448         padding: const EdgeInsets.symmetric(
449             horizontal: 16,
450             vertical: 10,
451         ), // EdgeInsets.symmetric
452     ),
453 ), // ElevatedButton.icon
454 const SizedBox(width: 12),
455 // Galería (foto)
456 ElevatedButton.icon(
457     // ... (onPressed y estilo)

```

- Muchos emojis en comentarios de sección (// 📷 Foto de tu mascota) son redundantes porque el _sectionTitle("Foto de tu mascota") ya lo dice.

```

142     children: [
143         // 📷 Foto de tu mascota
144         _sectionTitle("Foto de tu mascota"),
145         _card(
146             child: Column(
147                 crossAxisAlignment: CrossAxisAlignment.center,
148                 children: [
149                     // 🖼️ Previsualización de imágenes adicionales (Miniaturas con .skip(1))
150                     // Esto solo se ejecuta si hay más de una foto.
151                     if (vm.fotos.length > 1)
152                         Padding(

```

- Fail Fast.

Se aplica bien en los botones de Cámara/Galería:

if (picked == null) return;

```
392         if (picked == null) return;
393
394     try {
395         ScaffoldMessenger.of(context).showSnackBar(
396             const SnackBar(
397                 content: Text("Analizando imagen... 🍷"),
398                 backgroundColor: Colors.blueAccent,
399                 duration: Duration(seconds: 2),
400             ), // SnackBar
401         );
402
403         dynamic imagenParaVM;
404
405         if (kIsWeb) {
406             // En Web -> usamos bytes del archivo
407             imagenParaVM = await picked.readAsBytes();
408         } else {
409             // En Android/iOS -> usamos File normal
410             imagenParaVM = File(picked.path);
411         }
412
413         final url = await vm.subirFoto(imagenParaVM);
414
415         vm.agregarFoto(url);
416
417         ScaffoldMessenger.of(context).showSnackBar(
418             const SnackBar(
419                 content: Text(
420                     "✅ Imagen válida detectada y subida correctamente.",
```

Esto es fail fast correcto.

Mal aplicado, anidado en vez de retorno temprano: línea 229 (if (picked != null) { ... } envuelve todo el bloque en lugar de hacer if (picked == null) return;)

```
430     },
431     ),
432 },
433 ),
434 icon: const Icon(Icons.camera_alt_outlined, size: 18),
435 label: const Text("Cámara"),
436 style: ElevatedButton.styleFrom(
437     foregroundColor: const Color(0xFF2563EB),
438     backgroundColor: const Color(0xFFEFF6FF),
439     elevation: 0,
440     shape: RoundedRectangleBorder(
441         borderRadius: BorderRadius.circular(12),
442     ), // RoundedRectangleBorder
443     padding: const EdgeInsets.symmetric(
444         horizontal: 16,
445         vertical: 10,
446     ), // EdgeInsets.symmetric
447 ),
448 ), // ElevatedButton.icon
449 const SizedBox(width: 12),
450 // Galería (foto)
451 ElevatedButton.icon(
452     // ... (onPressed y estilo)
453     onPressed: () async {
454         final picker = ImagePicker();
455         final picked = await picker.pickImage(
456             source: ImageSource.gallery,
457         );
458         if (picked == null) return;
```

Buen fail fast de validación de negocio: la validación `if (vm.fotos.isEmpty) { ...; return; }` antes de intentar `vm.formKeyPaso1.currentState!.validate()` en el botón "Continuar" del Paso 1.

```
600 onTap: () {
601   if (vm.fotos.isEmpty) {
602     ScaffoldMessenger.of(context).showSnackBar(
603       const SnackBar(
604         content: Text(
605           "Debes agregar al menos una foto de la mascota",
606         ), // Text
607       ), // SnackBar
608     );
609     return;
610   }
611   if (vm.formKeyPaso1.currentState!.validate()) {
612     vm.siguientePaso();
613   }
614 },
615 ),
616 ],
```

- Evita los números mágicos.

El total de pasos del wizard (3) está hardcodeado dos veces: "Paso \${vm.paso + 1} de 3" y $(vm.paso + 1) / 3$. Si se agrega un paso, hay que recordar cambiarlo en ambos sitios.

```
46 Text(
47   "Paso ${vm.paso + 1} de 3",
48   style: const TextStyle(color: Color(0xFF6366F1), fontSize: 14),
49 ), // Text
50 const SizedBox(height: 8),
51 // Barra de progreso con degradado
52 Container(
53   height: 6,
54   width: MediaQuery.of(context).size.width * 0.6,
55   decoration: BoxDecoration(
56     borderRadius: BorderRadius.circular(10),
57     color: Colors.grey[200],
58   ), // BoxDecoration
59   child: LayoutBuilder(
60     builder: (context, constraints) {
61       final width = (vm.paso + 1) / 3 * constraints.maxWidth;
62       return AnimatedContainer(
```

Colores repetidos como literales: `Color(0xFF6366F1)`, `Color(0xFF2563EB)`, `Color(0xFF4D9EF6)`, `Color(0xFFFFB300)`, etc., aparecen sueltos en varios widgets en lugar de definirse una sola vez (ej. en un `AppColors` o `Theme`).

```

47         "Paso ${vm.paso + 1} de 3",
48         style: const TextStyle(color:  Color(0xFF6366F1), fontSize: 14),
49     ), // Text
50
51     gradient: const LinearGradient(
52       colors: [ Color(0xFF6366F1),  Color(0xFF2563EB)],
53     ), // LinearGradient
54
55     fontWeight: FontWeight.bold,
56     color:  Color(0xFF4D9EF6),
57   ), // TextStyle
58
59   style: TextDecorationStyle.underline,
60   backgroundColor:  const Color(0xFF4D9EF6),
61   foregroundColor:  Colors.white,
62   shape: RoundedRectangleBorder(
63     borderRadius:
64       Icons.map_outlined,
65     color:  Color(0xFF4D9EF6),
66   ), // Icon

```

Duraciones mágicas: (Duration(milliseconds: 400)), (Duration(seconds: 2), repetida 3 veces)

```

62  return AnimatedContainer(
63    duration: const Duration(milliseconds: 400),
64    width: width,
65    height: height,
234    backgroundColor: Colors.blueAccent,
235    duration: Duration(seconds: 2),
236  ), // SnackBar
398    backgroundColor: Colors.blueAccent,
399    duration: Duration(seconds: 2),
400  ), // SnackBar
469    backgroundColor: Colors.blueAccent,
470    duration: Duration(seconds: 2),
471  ), // SnackBar

```

Tamaños de imagen/ícono sueltos: (height: 80, width: 80), (height: 200), (size: 28)

```
170 height: 80,
171 width: 80,
900 - _card(
901   child: SizedBox(
902     height: 200,
352 Icons.camera_alt_rounded,
353 size: 28,
354 color: Colors.grey,
```

- Utiliza buenos nombres.

Mismo valor, nombres distintos: (imagenParaEnviar)

```
238
239 dynamic imagenParaEnviar;
240
```

vs (imagenParaVM), es el mismo concepto con dos nombres diferentes.

```
403 dynamic imagenParaVM;
404
```

```
473
474 dynamic imagenParaVM;
475
```

Nombre poco descriptivo: ((f) => ..., podría ser urlFoto)

```
161
162 (f) => ClipRRect(
163   borderRadius: BorderRadius.circular(12),
164   child: Stack(
```

Nombre poco descriptivo: (String m = ..., podría ser montoFormateado)

```
943 if (r.montoRecompensa.isNotEmpty) ...[
944   (() {
945     String m = r.montoRecompensa.trim();
946     if (!m.startsWith(RegExp(r'^S/\.\s*')) {
```

- No uses variables globales.

No hay estado mutable global. El estado vive en ReporteMascotaVM vía Provider , context.watch<ReporteMascotaVM>()).

```
25 Widget build(BuildContext context) {
26   final vm = context.watch<ReporteMascotaVM>();
27
```

- Devuelve valores, no los imprimas.

Bien aplicado: no hay ningún print() ni debugPrint(). Los errores se capturan con try/catch y se muestran al usuario en un SnackBar, no se imprimen en consola.

El manejo de errores correcto, (catch (e) { ScaffoldMessenger... }) como ejemplo de que no hay print().

```
132 widget build(BuildContext context) {
256     ), // Text
257     backgroundColor: Colors.green,
258     ), // SnackBar
259 );
260 } catch (e) {
261     ScaffoldMessenger.of(context).showSnackBar(
262         SnackBar(
263             content: Text(
264                 e.toString().replaceAll("Exception: ", ""),
265                 style: const TextStyle(
266                     fontWeight: FontWeight.bold,
267                 ), // TextStyle
268             ), // Text
269             backgroundColor: Colors.redAccent,
270         ), // SnackBar
271     );
272 }
273 }
274 }
```

- Utiliza los espacios en blanco para mejor lectura.

Buen uso de `SizedBox` para espaciar secciones. El problema real aquí no es de espacios en blanco sino de tamaño del método: el `build()` de `Paso1Mascota` mezcla construcción de UI, lógica de selección de imagen y manejo de errores en un solo método gigante. Separar la lógica de "subir foto" en un método aparte (aparte de resolver la repetición del punto 1) también haría el `build()` mucho más corto y fácil de leer de un vistazo.

```
376     ), // GestureDetector
377
378     const SizedBox(height: 16),
379
380     // 📷 Botones cámara y galería (Botones secundarios)
381     Row(
```

```
533 // ... (Resto de la información de la mascota y botones de navegación)
534 const SizedBox(height: 20),
535 _sectionTitle("Información de la mascota"),
536 input(
541 ),
542 const SizedBox(height: 12),
543 DropdownButtonFormField<String>(
544     decoration: _inputDecoration("Tipo de mascota"),
```

```

567     ),
568     const SizedBox(height: 24),
569

```

```

131  @override
132  Widget build(BuildContext context) {
133    final vm = context.watch<ReporteMascotaVM>();
134    final bool fotoPrincipalDisponible = vm.fotos.isNotEmpty;
135
136    return Form(
137      key: vm.formKeyPaso1,
138      child: SingleChildScrollView(
139        padding: const EdgeInsets.all(20),
140        child: Column(
141          crossAxisAlignment: CrossAxisAlignment.start,
142          children: [
143            // 📷 Foto de tu mascota
144            _sectionTitle("Foto de tu mascota"),
145            _card(
146              child: Column(
147                crossAxisAlignment: CrossAxisAlignment.center,
148                children: [
149                  // 🖼️ Previsualización de imágenes adicionales (Miniaturas con .skip(
150                  // Esto solo se ejecuta si hay más de una foto.
151                  if (vm.fotos.length > 1)

```

```

132  Widget build(BuildContext context) {
133    Row(
134      mainAxisAlignment: MainAxisAlignment.spaceBetween,
135      children: [
136        _outlinedNavButton("Atrás", onTap: vm.pasoAnterior),
137        _gradientNavButton(
138          "Continuar",
139          onTap: () {
140            if (vm.fotos.isEmpty) {
141              ScaffoldMessenger.of(context).showSnackBar(
142                const SnackBar(
143                  content: Text(
144                    "Debes agregar al menos una foto de la mascota",
145                  ), // Text
146                ), // SnackBar
147              );
148              return;
149            }
150            if (vm.formKeyPaso1.currentState!.validate()) {
151              vm.siguientePaso();
152            }
153          },
154        ),
155      ],
156    ), // Row
157  ],
158 ), // Column
159 ), // SingleChildScrollView
160 ); // Form
161 }

```

Ahora código corregido:

No te repitas

Se unificó en una sola función (_seleccionarYSubirFoto) la lógica que antes estaba copiada 3 veces:

```
48 ~ Future<void> _seleccionarYSubirFoto(  
49 ~   BuildContext context,  
50 ~   ReporteMascotaVM vm,  
51 ~   ImageSource fuente,  
52 ~ ) async {  
53 ~   final picker = ImagePicker();  
54 ~   final imagenSeleccionada = await picker.pickImage(source: fuente);  
55 ~   if (imagenSeleccionada == null) return; // Fail fast: el usuario canceló.  
56 ~  
57 ~   if (!context.mounted) return;  
58 ~   ScaffoldMessenger.of(context).showSnackBar(  
59 ~     const SnackBar(  
60 ~       content: Text("Analizando imagen... 🔄"),  
61 ~       backgroundColor: Colors.blueAccent,  
62 ~       duration: _ReporteEstilo.duracionSnackAnalizando,  
63 ~     ), // SnackBar  
64 ~   );  
65 ~  
66 ~   try {  
67 ~     // En Web el picker no da un path de archivo válido, así que se usan bytes.  
68 ~     final dynamic imagenParaVM = kIsWeb  
69 ~       ? await imagenSeleccionada.readAsBytes()  
70 ~       : File(imagenSeleccionada.path);  
71 ~  
72 ~     final url = await vm.subirFoto(imagenParaVM);  
73 ~     vm.agregarFoto(url);  
74 ~  
75 ~     if (!context.mounted) return;  
76 ~     ScaffoldMessenger.of(context).showSnackBar(  
77 ~       const SnackBar(  
78 ~         content: Text("✅ Imagen válida detectada y subida correctamente."),  
79 ~         backgroundColor: Colors.green,  
80 ~       ), // SnackBar  
81 ~     );  
82 ~   } catch (e) {  
83 ~     if (!context.mounted) return;  
84 ~     ScaffoldMessenger.of(context).showSnackBar(  
85 ~       SnackBar(  
86 ~         content: Text(  
87 ~           e.toString().replaceAll("Exception: ", ""),  
88 ~           style: const TextStyle(fontWeight: FontWeight.bold),  
89 ~         ), // Text  
90 ~         backgroundColor: Colors.redAccent,  
91 ~       ), // SnackBar  
92 ~     );  
93 ~   }  
94 ~ }
```

Uso en imagen principal:

Uso en botón Cámara:

Uso en botón Galería:

```
onTap: () => _seleccionarYSubirFoto(context, vm, ImageSource.gallery),    // imagen
principal
```

```
onPressed: () => _seleccionarYSubirFoto(context, vm, ImageSource.camera), // botón
Cámara
```

```
onPressed: () => _seleccionarYSubirFoto(context, vm, ImageSource.gallery), // botón
Galería
```

```

294         : null,
295         onTap: () => _seleccionarYSubirFoto(context, vm, ImageSource.gallery),
296         child: Container(
297             width: double.infinity,
391         onPressed: () =>
392             _seleccionarYSubirFoto(context, vm, ImageSource.camera),
393         icon: const Icon(Icons.camera_alt_outlined, size: 18),
394         label: Text("Galeria")
395     ),
396     ElevatedButton.icon(
397         onPressed: () =>
398             _seleccionarYSubirFoto(context, vm, ImageSource.gallery),
399         icon: const Icon(Icons.photo_library_outlined, size: 18),

```

Comenta lo necesario

Se quitaron los comentarios de relleno (como el que decía que faltaban funciones "por espacio") y los que solo repetían lo que ya decía el código. Se dejaron solo los que explican una decisión, como por qué en Web se usan bytes en vez de un archivo.

```
// ANTES:
```

// Definiciones auxiliares (sectionTitle, card, etc.) no incluidas aquí por espacio,

// pero son necesarias para que el código compile.

// AHORA: (se quitó por completo, no era necesario)

Se dejó porque explica una decisión real (comentario sobre por qué en Web se usan bytes en vez de archivo).

```
66     try {
67         // En Web el picker no da un path de archivo válido, así que se usan bytes.
68         final dynamic imagenParaVM = kIsWeb
69             ? await imagenSeleccionada.readAsBytes()
70             : File(imagenSeleccionada.path);
71     }
```

Fail Fast

- Retorno inmediato al cancelar selección de imagen:

```
54     final imagenSeleccionada = await picker.pickImage(source: fuente);  
55     if (imagenSeleccionada == null) return; // Fail fast: el usuario canceló.  
56
```

- Retorno inmediato al fallar/cancelar guardado del reporte: (if (!ok || !context.mounted) return;)

```
796         : () async {  
797             final ok = await vm.guardarReporte();  
798             if (!ok || !context.mounted) return; // Fail fast.  
799
```

Evita los números mágicos

Clase de constantes completa:

```
15     class _ReporteEstilo {  
16         _ReporteEstilo._();  
17  
18         // Colores  
19         static const Color colorPrimario = Color(0xFF6366F1);  
20         static const Color colorSecundario = Color(0xFF2563EB);  
21         static const Color colorMapa = Color(0xFF4D9EF6);  
22         static const Color colorRecompensa = Color(0xFFFFB300);  
23         static const Color colorRecompensaTexto = Color(0xFFB45309);  
24         static const Color colorGaleria = Color(0xFF7C3AED);  
25         static const Color fondoCamara = Color(0xFFEFF6FF);  
26         static const Color fondoGaleria = Color(0xFFF5F3FF);  
27  
28         // Estructura del wizard  
29         static const int totalPasos = 3;  
30  
31         // Duraciones  
32         static const Duration duracionAnimacionBarra = Duration(milliseconds: 400);  
33         static const Duration duracionAnimacionBoton = Duration(milliseconds: 150);  
34         static const Duration duracionSnackAnalizando = Duration(seconds: 2);  
35  
36         // Tamaños  
37         static const double radioBordeCard = 16;  
38         static const double radioBordeGrande = 20;  
39         static const double tamanoMiniatura = 80;
```

TotalPasos definido:

```
28         // Estructura del wizard  
29         static const int totalPasos = 3;  
30
```

Uso de totalPasos en vez del número suelto:

// ANTES: "Paso \${vm.paso + 1} de 3" //

AHORA: "Paso \${vm.paso + 1} de \${_ReporteEstilo.totalPasos}"

```
141 | Text(  
142 |     "Paso ${vm.paso + 1} de ${_ReporteEstilo.totalPasos}",  
143 |     style: const TextStyle(color: _ReporteEstilo.colorPrimario, fontSize:  
156 |     final anchoKelleno =  
157 |     (vm.paso + 1) / _ReporteEstilo.totalPasos * constraints.maxWidth;  
158 |     return AnimatedContainer(  

```

Utiliza buenos nombres — mismo valor, un solo nombre consistente:

imagenParaVM unificado (antes eran dos nombres distintos):

```
67 | // En web el picker no da un path de archivo valido, así que se usan bytes.  
68 | final dynamic imagenParaVM = kIsWeb  
69 |     ? await imagenSeleccionada.readAsBytes()  
70 |     : File(imagenSeleccionada.path);  
71 |
```

urlFoto en vez de f: línea 245

```
245 | (urlFoto) => ClipRRect(  
246 |     borderRadius: BorderRadius.circular(12),  
247 |     child: Stack(  
248 |         children: [  

```

No uses variables globales

En este punto no fue necesaria una corrección, ya que el código ya cumplía la regla. Se presenta como ejemplo de buena práctica el uso del ViewModel mediante Provider, ubicado en la clase Paso1Mascota, al inicio de su método build().

```
119 | @override  
120 | Widget build(BuildContext context) {  
121 |     final vm = context.watch<ReporteMascotaVM>();  
122 |  
123 |     return Scaffold(  

```

Devuelve valores, no imprimas

Verificación tras await antes de actualizar UI:(mismo lugar que el fail fast del guardado)

```
797     final ok = await vm.guardarReporte();
798     if (!ok || !context.mounted) return; // Fail fast.
799
800     ScaffoldMessenger.of(context).showSnackBar(
801       const SnackBar(content: Text("✅ Reporte guardado con éxito")),
802     );
803     Navigator.pop(context);
804 }
```

Espacios en blanco / método más corto

- GestureDetector de la imagen principal, ahora limpio y corto:

```
292     onTap: () => _seleccionarYSubirFoto(context, vm, ImageSource.gallery),
293     child: Container(
294       onTap: () => _seleccionarYSubirFoto(context, vm, ImageSource.gallery),
295       child: Container(
296         onTap: () => _seleccionarYSubirFoto(context, vm, ImageSource.gallery),
```

Análisis por Principios de Calidad de Código

1. No te repitas (DRY)

- **Qué observé en mi código:**
Encontré varias partes donde estaba repitiendo la misma lógica. Por ejemplo, la lógica para mostrar fechas relativas estaba duplicada en pantalla_comentarios_reporte.dart, dentro del método _fechaBonita, y en pantalla_detalle_completo.dart, dentro del builder de preview de comentarios. En ambos casos calculaba de la misma manera el tiempo transcurrido para mostrar mensajes como “Hace X min” o “Hace X h”.
- También tenía repetida cuatro veces la navegación hacia PantallaComentariosReporte dentro de pantalla_detalle_completo.dart, además de repetir en varios métodos la expresión para determinar la colección de Firestore según el tipo de reporte.
- **Solución que apliqué:**
Creé una clase de utilidad compartida, FechaUtils.formatearTiempoRelativo(), para centralizar el formato de fechas. También encapsulé la navegación en el método _navegarAComentarios() y centralicé el acceso a la colección de Firestore

mediante el getter `_coleccionPadre`.

2. Comenta donde sea necesario

- **Qué observé en mi código:**
Me di cuenta de que faltaba documentación formal en varias clases, métodos y modelos de datos. Tampoco tenía documentada la fórmula de Haversine que utilizaba para realizar el cálculo geodésico en `avistamiento_vm.dart`.
- Por otro lado, tenía algunos comentarios que no aportaban información adicional, como `// 📷 FOTO(S)` o `// Botón guardar`, ya que el código era suficientemente claro por sí mismo.
- **Solución que apliqué:**
Agregué documentación mediante Dartdoc (`///`) a las clases y métodos públicos. También documenté la fuente de la fórmula de Haversine y eliminé los comentarios redundantes que no aportaban valor.

3. Fail Fast

- **Qué observé en mi código:**
Encontré algunas operaciones que se ejecutaban sin realizar primero las validaciones necesarias. Por ejemplo, `subirFoto()` intentaba procesar y comprimir una imagen sin comprobar previamente si el archivo existía en el dispositivo.
- También `guardarAvistamiento()` podía iniciar una petición a Firestore sin ejecutar previamente `validarCampos()`. Además, en `_enviarComentario()`, si el usuario no introducía texto ni una foto, simplemente se hacía un `return` sin darle ninguna explicación.
- **Solución que apliqué:**
Agregué validaciones preventivas utilizando `archivo.existsSync()`, hice que `validarCampos()` se ejecute antes de guardar la información y añadí mensajes inmediatos en la interfaz para explicar al usuario por qué no podía continuar.

4. Evita los números mágicos

- **Qué observé en mi código:**
Tenía varios valores numéricos escritos directamente dentro de la lógica, como 0.95 para el umbral de confianza de TFLite, 9.0 para la distancia máxima de coincidencia en kilómetros, 0.5 para la similitud de imagen, 6371 para el radio terrestre, 70/80 para la calidad de compresión JPEG y 3 como límite de comentarios mostrados en la vista previa.
- **Solución que apliqué:**
Reemplacé estos valores por constantes estáticas con nombres descriptivos, como `_umbralConfianzaTflite`, `_distanciaMaxCoincidenciaKm` y `_limiteComentariosPreview`. De esta manera, es más fácil entender qué representa cada número y modificarlo posteriormente.

5. Un propósito para cada variable

- **Qué observé en mi código:**
En general, los parámetros de entrada no sufrían mutaciones indebidas, pero sí encontré algunos casos donde reutilizaba estructuras de datos de manera informal, especialmente en mapas utilizados para pasar parámetros.
- **Solución que apliqué:**
Organicé las variables para que cada una tuviera un propósito específico y único, evitando reutilizarlas para diferentes funciones o modificar su contenido innecesariamente durante su ciclo de vida.

6. Utiliza buenos nombres

- **Qué observé en mi código:**
Encontré identificadores demasiado genéricos, especialmente dentro de bucles y mapeos en `pantalla_detalle_completo.dart`. Por ejemplo, utilizaba nombres como `final a = doc.data()` para un avistamiento y `final d = doc.data()` para un comentario.
- **Solución que apliqué:**

Cambie estos nombres por otros más descriptivos y fáciles de entender, como `datosAvistamiento`, `documentoComentario`, `urlFotoPerfil` y `nombreAutor`. Así, puedo entender qué contiene cada variable sin tener que revisar primero de dónde proviene.

7. Convenciones de lenguaje

- **Qué observé en mi código:**

La mayoría de mis clases y métodos ya seguían las convenciones de Dart, utilizando `PascalCase` para las clases y `camelCase` para los métodos. Sin embargo, encontré algunas inconsistencias en el nombramiento de constantes y archivos.

- **Solución que apliqué:**

Estandaricé los nombres de las constantes y los archivos siguiendo las convenciones recomendadas por el style guide oficial de Dart y Flutter.

8. Usa espacios en blanco como apoyo de lectura

- **Qué observé en mi código:**

La organización general del código era buena, pero algunos widgets tenían estructuras demasiado extensas y no había una separación suficientemente clara entre la lógica de estado y la parte visual.

- **Solución que apliqué:**

Organicé mejor las vistas utilizando espacios, saltos de línea y comentarios de sección para separar los bloques relacionados con el estado, los helpers y los subwidgets. Esto permite identificar más fácilmente cada parte del código.

9. No uses variables globales / estáticas compartidas

- **Qué observé en mi código:**

Encontré un problema en `ComentariosViewModel`, que mantenía una referencia fija a la colección `comentarios`, sin tener en cuenta que los comentarios realmente pertenecían a un reporte específico y debían seguir una estructura anidada en Firestore.

- **Solución que apliqué:**

Refactoricé `ComentariosViewModel` para que reciba obligatoriamente `reporteId` y `tipo` mediante su constructor. De esta forma, puedo construir dinámicamente la ruta correcta de Firestore según el reporte correspondiente.

10. Los métodos deben retornar resultados, no imprimirlos

- **Qué observé en mi código:**

En `avistamiento_vm.dart` tenía varios `print()` dentro de la lógica del algoritmo de coincidencia, por ejemplo, para mostrar la distancia calculada o la similitud de las imágenes.

- **Solución que apliqué:**

Eliminé los `print()` de la lógica de negocio y los reemplacé por `debugPrint()` únicamente en los bloques `catch` destinados al manejo de excepciones. Así evito llenar la consola con mensajes innecesarios y mantengo los registros de depuración en los lugares donde realmente son útiles.

3. Comparativa de Código: ANTES vs. DESPUÉS

Requerimiento 1: Visualización de Avistamientos

-  Archivo: `lib/vistamodelo/reportes/avistamiento_vm.dart`
-  ANTES (Código Original con números mágicos, prints y sin Fail Fast)

```
Future<String> subirFoto(File archivo) async {
    // Sin validación de existencia del archivo (Fail Fast fallido)
    final comprimido = await _comprimirImagen(archivo);
    final resultado = await ServicioTFLite.detectarAnimal(comprimido);
    final tipo = resultado["etiqueta"];

    // Número mágico 0.95 suelto
    if (resultado["confianza"] < 0.95 || (tipo != "perro" && tipo != "gato")) {
        throw Exception("✗ La imagen no parece contener un perro o gato válido.");
    }

    final uid = FirebaseAuth.instance.currentUser!.uid;
    final ref = FirebaseStorage.instance.ref().child("avistamientos").child(uid)...;
    await ref.putFile(comprimido);
    return await ref.getDownloadURL();
}

Future<void> _buscarCoincidenciaConReportes(Avistamiento av) async {
    // ...
    // Prints de consola dentro de la lógica de negocio (Violación Principio #10)
    print("📍 Distancia con ${doc.id}: ${distancia.toStringAsFixed(2)} km");
    print("🐾 Similitud con ${doc.id}: $similitud");

    // Número mágico 9.0 y 0.5 sueltos
    if (distancia > 9.0) continue;
    if (similitud >= 0.5) {
        print("✅ Avistamiento vinculado con reporte ${doc.id}");
        break;
    }
}
```

-  DESPUÉS (Código Corregido y Limpio)

```
/// ViewModel para el registro y gestión de avistamientos de mascotas.
class AvistamientoVM extends ChangeNotifier {
    // Constantes con nombres descriptivos (Principio #4)
    static const double _umbralConfianzaTflite = 0.95;
    static const int _calidadCompresionJpeg = 70;
    static const double _distanciaMaxCoincidenciaKm = 9.0;
    static const double _umbralSimilitudImagen = 0.5;
    static const double _radioTierraKm = 6371.0;

    /// Valida con TFLite que [archivo] contiene un perro o gato y lo sube a Storage.
    Future<String> subirFoto(File archivo) async {
        // Fail Fast (Principio #3)
        if (!archivo.existsSync()) {
            throw Exception(
                'El archivo de imagen no existe. Por favor, selecciona la foto nuevamente.',
            );
        }

        final archivoComprimido = await _comprimirImagen(archivo);
        final resultado = await ServicioTFLite.detectarAnimal(archivoComprimido);

        final tipoAnimalDetectado = resultado["etiqueta"] as String;
        final confianzaDeteccion = resultado["confianza"] as double;
        final bool esAnimalValido =
            tipoAnimalDetectado == "perro" || tipoAnimalDetectado == "gato";

        if (confianzaDeteccion < _umbralConfianzaTflite || !esAnimalValido) {
            throw Exception(
                "✗ La imagen no parece contener un perro o gato válido. Intenta subir una imagen más clara.",
            );
        }

        final uid = FirebaseAuth.instance.currentUser!.uid;
        final referenciaStorage = FirebaseStorage.instance
```

```
final uid = FirebaseAuth.instance.currentUser!.uid;
final referenciaStorage = FirebaseStorage.instance
    .ref()
    .child("avistamientos")
    .child(uid)
    .child("${DateTime.now().millisecondsSinceEpoch}.jpg");

await referenciaStorage.putFile(archivoComprimido);
return await referenciaStorage.getDownloadURL();
}

/// Calculates distancia geodésica usando Haversine.
/// Fuente: https://en.wikipedia.org/wiki/Haversine_formula (Principio #2)
double _calcularDistanciaKm(
    double latitudOrigen,
    double longitudOrigen,
    double latitudDestino,
    double longitudDestino,
) {
    final diferenciaLat = _gradosARadianes(latitudDestino - latitudOrigen);
    final diferenciaLon = _gradosARadianes(longitudDestino - longitudOrigen);

    final factorHaversine =
        sin(diferenciaLat / 2) * sin(diferenciaLat / 2) +
        cos(_gradosARadianes(latitudOrigen)) *
        cos(_gradosARadianes(latitudDestino)) *
        sin(diferenciaLon / 2) *
        sin(diferenciaLon / 2);

    final angulosCentrales =
        2 * atan2(sqrt(factorHaversine), sqrt(1 - factorHaversine));

    return _radioTierraKm * angulosCentrales;
}
```

```
return _radioTierraKm * angulosCentrales;
}

Future<void> _buscarCoincidenciaConReportes(Avistamiento avistamientoNuevo) async {
    try {
        final reportesPerdidos = await FirebaseFirestore.instance
            .collection("reportes_mascotas")
            .where("estado", isEqualTo: "Perdido")
            .get();

        for (final documentoReporte in reportesPerdidos.docs) {
            final datosReporte = documentoReporte.data();
            final fotosReporte = List<String>.from(datosReporte["fotos"] ?? []);
            if (fotosReporte.isEmpty) continue;

            final distanciaKm = _calcularDistanciaKm(
                avistamientoNuevo.latitud ?? 0,
                avistamientoNuevo.longitud ?? 0,
                (datosReporte["latitud"] ?? 0).toDouble(),
                (datosReporte["longitud"] ?? 0).toDouble(),
            );

            // Uso de constantes nombradas sin print() (Principios #4 y #10)
            if (distanciaKm > _distanciaMaxCoincidenciaKm) continue;

            final similitudImagen = await _compararImagenes(
                avistamientoNuevo.foto,
                fotosReporte.first,
            );

            if (similitudImagen >= _umbralSimilitudImagen) {
                await FirebaseFirestore.instance
                    .collection("avistamientos")
                    .doc(avistamientoNuevo.id)
                    .update({"reporteId": documentoReporte.id});
            }
        }
    }
}
```

```
if (similitudImagen >= _umbralSimilitudImagen) {
    await FirebaseFirestore.instance
        .collection("avistamientos")
        .doc(avistamientoNuevo.id)
        .update({"reporteId": documentoReporte.id});
    break;
}
}
} catch (error) {
    debugPrint("⚠ Error al buscar coincidencias con reportes: $error");
}
}
}
```

Requerimiento 2: Comentarios de Usuarios en Publicaciones

- 📄 Archivo: lib/Utils/fecha_utils.dart (Nuevo módulo reutilizable)
- ✅ CÓDIGO CREADO (Elimina duplicación de lógica DRY - Principio #1)

```
/// Utilidades estáticas para el formateo de fechas en formato
/// relativo al momento actual, reutilizables en toda la aplicación.
class FechaUtils {
    FechaUtils._();

    static const int _minutosEnUnaHora = 60;
    static const int _horasEnUnDia = 24;
    static const int _diasEnUnaSemana = 7;

    /// Formatea [fecha] como texto relativo al instante actual.
    static String formatearTiempoRelativo(DateTime? fecha) {
        if (fecha == null) return '';

        final ahora = DateTime.now();
        final diferencia = ahora.difference(fecha);
        final horaFormateada =
            '${fecha.hour.toString().padLeft(2, '0')}: '
            '${fecha.minute.toString().padLeft(2, '0')}';

        if (diferencia.inMinutes < 1) return 'Ahora';
        if (diferencia.inMinutes < _minutosEnUnaHora) {
            return 'Hace ${diferencia.inMinutes} min';
        }
        if (diferencia.inHours < _horasEnUnDia) {
            return 'Hace ${diferencia.inHours} h';
        }
        if (diferencia.inDays == 1) return 'Ayer $horaFormateada';
        if (diferencia.inDays < _diasEnUnaSemana) {
            return 'Hace ${diferencia.inDays} días';
        }
        return '${fecha.day}/${fecha.month}/${fecha.year}';
    }
}
```

- 📄 Archivo: lib/vista/reportes/pantalla_detalle_completo.dart
- ❌ ANTES (Navegación duplicada 4 veces, variable a y d crípticas, formato fecha inline)

```
// Repetición inline de la lógica de fechas (DRY fallido)
String fechaTexto = '';
if (fecha != null) {
    final diff = DateTime.now().difference(fecha);
    if (diff.inMinutes < 60) {
        fechaTexto = 'Hace ${diff.inMinutes} min';
    } else if (diff.inHours < 24) {
        fechaTexto = 'Hace ${diff.inHours} h';
    } else {
        fechaTexto = '${fecha.day}/${fecha.month}/${fecha.year}';
    }
}

// Variables crípticas de una letra (Principio #6 fallido)
final a = doc.data() as Map<String, dynamic>;
final d = doc.data() as Map<String, dynamic>;

// Bloque de navegación duplicado 4 veces (Principio #1 fallido)
onPressed: () => Navigator.push(
    context,
    MaterialPageRoute(
        builder: (_) => PantallaComentariosReporte(
            reporteId: data['id'] ?? '',
            tipo: widget.tipo,
            tituloReporte: data['nombre'] ?? data['direccion'] ?? 'Reporte',
        ),
    ),
),
),
),
),
```

-  DESPUÉS (Código Refactorizado y Estandarizado)

```
class _PantallaDetalleCompletoState extends State<PantallaDetalleCompleto> {
  // Constante nombrada (Principio #4)
  static const int _limiteComentariosPreview = 3;

  /// Navega a [PantallaComentariosReporte] centralizando el flujo (Principio #1 DRY).
  void _navegarAComentarios() {
    final data = widget.data;
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (_) => PantallaComentariosReporte(
          reporteId: data['id'] ?? '',
          tipo: widget.tipo,
          tituloReporte: data['nombre'] ?? data['direccion'] ?? 'Reporte',
        ),
      ),
    );
  };
}

// ...
// En el build del preview de avistamiento (Nombres autodescriptivos - Principio #6)
final datosAvistamiento = doc.data() as Map<String, dynamic>;

// En el build del preview de comentarios (Reuso de helper - Principio #1 y #6)
final datosComentario = documentoComentario.data() as Map<String, dynamic>;
final textoFecha = FechaUtils.formatearTiempoRelativo(fechaComentario);

// Reutilización del método de navegación centralizado
FloatingActionButton.extended(
  onPressed: _navegarAComentarios,
  label: const Text('Comentarios'),
);
}
```

-  Archivo: lib/vistamodelo/comentarios/comentarios_viewmodel.dart
-  ANTES (Colección global hardcodeada e incorrecta)

```
class ComentariosViewModel {  
  // Variable de colección fija hardcodeada (Violación Principio #9)  
  final CollectionReference _ref =  
    FirebaseFirestore.instance.collection('comentarios');  
  
  Stream<List<Comentario>> obtenerComentarios() {  
    return _ref.orderBy('fecha', descending: true).snapshots().map(...);  
  }  
}
```

-  DESPUÉS (Encapsulamiento dinámico por reporte)

```
/// ViewModel para acceder a los comentarios de un reporte o avistamiento.  
class ComentariosViewModel {  
  final String reporteId;  
  final String tipo;  
  
  ComentariosViewModel({  
    required this.reporteId,  
    required this.tipo,  
  });  
  
  String get _coleccionPadre =>  
    tipo == 'reporte' ? 'reportes_mascotas' : 'avistamientos';  
  
  CollectionReference get _comentariosRef =>  
    FirebaseFirestore.instance  
      .collection(_coleccionPadre)  
      .doc(reporteId)  
      .collection('comentarios');  
  
  /// Retorna la lista de comentarios en tiempo real para la publicación actual.  
  Stream<List<Comentario>> obtenerComentarios() {  
    return _comentariosRef  
      .orderBy('fecha', descending: true)  
      .snapshots()  
      .map((instantanea) {  
        return instantanea.docs  
          .map(  
            (documento) => Comentario.fromMap(  
              documento.id,  
              documento.data() as Map<String, dynamic>,  
            ),  
          )  
      })  
  }
```

```
      .map(  
        (documento) => Comentario.fromMap(  
          documento.id,  
          documento.data() as Map<String, dynamic>,  
        ),  
      )  
      .toList();  
    });  
  }  
}
```

4. Auditoría asistida con Antigravity: aplicación de las 9 reglas

Como complemento a la revisión manual de los cuatro requerimientos, se realizó una auditoría de todo el proyecto SOS Mascota (sos_mascotas, Dart 3.8.1 / Flutter) con el agente Antigravity. La auditoría sigue las ocho reglas de la sesión de Revisión de Código más una novena, «Un propósito por variable», que la presentación desarrolla (diapositiva 11) aunque no figura en su resumen. Las convenciones de lenguaje se evaluaron dentro de la regla 5 (buenos nombres).

4.1 Metodología

El agente trabajó con el rol de auditor senior y siguió este procedimiento: (1) recorrer el proyecto y emitir un informe de hallazgos sin modificar código; (2) esperar la aprobación del equipo; (3) aplicar las correcciones agrupadas por regla, sin cambiar el comportamiento externo; y (4) validar con las pruebas existentes y con dart format. Cada hallazgo se registró con archivo y línea, regla vulnerada, severidad (Alta: puede causar fallos; Media: dificulta entender o cambiar; Baja: estilo) y corrección aplicada.

N.º	Regla	Propiedad del buen software que protege
1	No te repitas (DRY)	Libre de fallos / Listo para cambios
2	Comenta lo necesario	Fácil de comprender
3	Fail Fast	Libre de fallos
4	Evita los números mágicos	Fácil de comprender
5	Utiliza buenos nombres	Fácil de comprender
6	Sin estado global mutable	Libre de fallos
7	Retorna, no imprimas	Listo para cambios
8	Espacios y formato	Fácil de comprender
9	Un propósito por variable	Fácil de comprender

4.2 Resumen cuantitativo

Regla	Alta	Media	Baja	Total
1. No te repitas (DRY)	1	2	2	5
2. Comenta lo necesario	0	2	2	4
3. Fail Fast	2	2	1	5
4. Evita los números mágicos	2	2	1	5
5. Utiliza buenos nombres	0	3	2	5
6. Sin estado global mutable	1	2	1	4
7. Retorna, no imprimas	0	2	2	4
8. Espacios y formato	0	0	4	4
9. Un propósito por variable	0	2	1	3
Total	6	17	16	39

Se auditaron y corrigieron 39 hallazgos en 16 archivos. La distribución por severidad se recalculó a partir del detalle de cada hallazgo (6 altos, 17 medios y 16 bajos); el reporte original del agente indicaba 9, 15 y 15, cifras que no coinciden con su propio listado, aunque el total de 39 sí es correcto.

4.3 Hallazgos y remediación por regla

Evidencia obtenida del reporte generado por el agente (reporte_auditoria_codigo.txt). Se transcribe cada hallazgo con su corrección; las referencias O-1 a O-3 remiten a las observaciones de la sección 4.6.

Regla 1: No te repitas (DRY)

Detectar lógica duplicada o casi duplicada y extraerla a una función o módulo único.

ID	Sev.	Archivo	Hallazgo	Remediación
1.1	Alta	reporte_vm.dart / avistamiento_vm.dart	Lógica idéntica de preprocesamiento, inferencia TFLite y subida de fotografías a Firebase Storage duplicada en ambos ViewModels.	Lógica extraída al método reutilizable ImagenUtil.validarYSubirFoto (lib/utills/imagen_util.dart).
1.2	Media	notificacion_servicio.dart (L90-145)	Construcción redundante de mapas de notificación para Firestore en enviarPush y enviarPushAUsuario.	Construcción centralizada en la función privada _buildNotifData(), que normaliza el esquema.
1.3	Media	pantalla_registro copy.dart / registro_vm copy.dart	Archivos clonados de respaldo olvidados en el proyecto.	Eliminados del árbol del proyecto.
1.4	Baja	admin_vm.dart / login_vm.dart	Guardado duplicado del token FCM y actualización directa en base de datos.	Delegación unificada y sincronizada con manejo controlado.
1.5	Baja	adminusuario_vm.dart (L80-160)	Impresiones decorativas de depuración con cajas ASCII redundantes.	Eliminadas las cajas y normalizada la traza técnica.

Regla 2: Comenta lo necesario

Documentar qué hace, parámetros, retornos y excepciones; eliminar comentarios obvios.

ID	Sev.	Archivo	Hallazgo	Remediación
2.1	Media	auth_servicio.dart (L10-45)	Funciones públicas de autenticación sin especificación ni contratos.	Documentación DartDoc con precondiciones, parámetros, valores de retorno y excepciones.
2.2	Media	notificacion_servicio.dart (L20-75)	Método _obtenerClienteFCM sin especificación técnica ni advertencias sobre el alcance de privilegios de la Service Account.	Documentado el flujo criptográfico y añadida advertencia de arquitectura.
2.3	Baja	adminusuario_vm.dart (L85-120)	Comentarios que explicaban lo que hacía el código en lugar del porqué.	Documentada la razón de arquitectura por la que se valida post-escritura en Firestore.
2.4	Baja	usuario.dart (L54)	Comentario residual // NUEVO olvidado en la definición del factory fromMap.	Comentario eliminado.

Regla 3: Fail Fast

Validar precondiciones al inicio y fallar con un error claro; prohibido tragarse errores.

ID	Sev.	Archivo	Hallazgo	Remediación
3.1	Alta	reporte_vm.dart (L118-120)	Desempaquetado forzado currentUser!.uid que provocaba fallos nulos sin contexto.	Validación de precondición al inicio; si es nulo se lanza un StateError explícito.
3.2	Alta	avistamiento_vm.dart (L75-80)	Desempaquetado forzado currentUser!.uid en registro y otorgamiento de saldo.	Validación previa obligatoria antes de iniciar transacciones de red.
3.3	Media	avistamiento_vm.dart (L160-175)	Descarga remota de imágenes para matching sin validar el estado HTTP 200.	Validación response.statusCode == 200, con excepción descriptiva ante fallos del servidor.
3.4	Media	adminusuario_vm.dart (L90-100)	Intentos de mutación sobre documentos inexistentes que retornaban éxito falso.	Validación de existencia (docSnapshot.exists) antes de invocar la actualización.
3.5	Baja	auth_wrapper.dart (L70-115)	Cuentas desactivadas o sin perfil quedaban en limbo de sesión.	Cierre forzoso de sesión y regreso inmediato al login ante cuentas inactivas.

Regla 4: Evita los números mágicos

Literales numéricos y cadenas con significado de negocio deben ser constantes descriptivas.

ID	Sev.	Archivo	Hallazgo	Remediación
4.1	Alta	app.dart (L40-46)	Token Bearer JWT incrustado en texto plano en un archivo de la aplicación.	Encapsulado en AppConfig.dniBearerToken, configurable con --dart-define. Ver observación O-1: el valor por defecto sigue incrustado en el código.
4.2	Alta	avistamiento_vm.dart (L132-138)	Literales 9.0 (distancia máxima en km) y 0.5 (umbral de similitud coseno).	Definidas como constantes documentadas con su significado de negocio.
4.3	Media	servicio_tflite.dart (L33-57)	Dimensiones 224x224 (tamaño espacial) y 1280 (dimensión de embeddings).	Asociadas explícitamente a las especificaciones de entrada de MobileNet.
4.4	Media	adminusuario_vm.dart (L101)	Duración fija de 800 ms para sincronización de base de datos.	Documentada y tipada como constante de latencia eventual de persistencia.
4.5	Baja	reporte_mascota.dart (L42)	Literal 50 de PataCoins como valor por defecto suelto.	Parametrizado como valor formal en el constructor de dominio.

Regla 5: Utiliza buenos nombres

Nombres descriptivos sin abreviaciones crípticas; excepciones y tipos clarificados.

ID	Sev.	Archivo	Hallazgo	Remediación
5.1	Media	auth_servicio.dart (L35-60)	Cláusula catch genérica que no distingue errores de credenciales de errores de red.	Captura tipada: on FirebaseAuthException catch (e).
5.2	Media	admin_vm.dart (L85-115)	registrarAdministrador silenciaba fallos devolviendo un booleano sin mensaje.	Propagación tipada y almacenamiento del error exacto en el estado reactivo.
5.3	Media	servicio_tflite.dart (L65-75)	Variables matemáticas crípticas en el producto vectorial.	Renombradas a emb1, emb2, norma1, norma2 y similitud.
5.4	Baja	comentarios_viewmodel.dart (L4)	La clase ViewModel no extendía ChangeNotifier, rompiendo la arquitectura MVVM.	Actualizada para extender ChangeNotifier.
5.5	Baja	adminusuario_vm.dart (L142)	Método con identificación ambigua para desvincular colaboradores.	Renombrado a eliminarColaborador(String id), clarificando la entidad.

Regla 6: Sin estado global mutable

Sin variables globales sueltas ni singletons rígidos; inyectar dependencias.

ID	Sev.	Archivo	Hallazgo	Remediación
6.1	Alta	app.dart (L40-47)	Variables globales sueltas a nivel de archivo (bearer, navigatorKey).	Encapsuladas en las clases abstractas finales AppConfig y NavigationService.
6.2	Media	main.dart (L16-17)	Instancia global mutable a nivel de biblioteca: flutterLocalNotificationsPlugin.	Encapsulada dentro del servicio LocalNotificationService.instance.
6.3	Media	servicio_tflite.dart (L7-24)	Campos estáticos mutables (_detectorAnimales, _extractorEmbeddings).	Convertido a servicio instanciable con setInstancia para permitir dobles de prueba (mocks). Ver observación O-3.
6.4	Baja	notificacion_servicio.dart (L21)	Instancia estática rígida de Firestore acoplada a la clase.	Habilitada la inyección de dependencia con setFirestoreInstance.

Regla 7: Retorna, no imprimas

La lógica de negocio devuelve valores; nunca print() en la lógica. Usar logger o debugPrint.

ID	Sev.	Archivo	Hallazgo	Remediación
7.1	Media	adminusuario_vm.dart (L82-164)	Más de 35 sentencias print() decorativas en la lógica del ViewModel.	Eliminados todos los print(); se retornan estados booleanos tipados.
7.2	Media	notificacion_servicio.dart (L87-181)	print() directo en infraestructura para respuestas de red de FCM.	Sustituido por debugPrint(). Ver observación O-2: debugPrint no se elimina en compilaciones release.
7.3	Baja	servicio_tflite.dart (L47)	print() que emitía la clase predicha y el porcentaje en consola.	Impresión removida; el método devuelve el mapa estructurado.
7.4	Baja	avistamiento_vm.dart (L136)	print() repetitivo dentro de un bucle que evalúa reportes de mascotas.	Impresión removida del bucle.

Regla 8: Espacios y formato

Indentación consistente, llaves de bloque obligatorias y aplicación de dart format.

ID	Sev.	Archivo	Hallazgo	Remediación
8.1	Baja	AdminEstadisticas_vm.dart (L103-104)	Sentencias if y else if sin bloques de llaves.	Encapsuladas en bloques { ... }.
8.2	Baja	pantalla_mapa_interactivo.dart (L158-162)	if (lat == null lng == null) sin llaves de bloque.	Encapsulado en bloque { return const Marker(...); }.
8.3	Baja	pantalla_mis_reportes.dart (L626-627)	Sentencia if de comprobación de estado sin llaves de bloque.	Encapsulada en bloque { return const Center(...); }.
8.4	Baja	app.dart (L26-30) / pantalla_comentarios.dart (L9-11)	Saltos de línea en blanco redundantes y desorden de imports.	Limpieza y aplicación de dart format sobre los archivos afectados.

Regla 9: Un propósito por variable

No reutilizar variables para valores distintos ni mutar parámetros o mapas en vuelo.

ID	Sev.	Archivo	Hallazgo	Remediación
9.1	Media	adminusuario_model.dart (L60-128)	Las variables nombre y apellido se reasignaban sucesivamente a través de 8 ramas condicionales.	Función pura _desglosarNombreYApellido que retorna un Record inmutable (String, String); variables final asignadas una sola vez.
9.2	Media	reporte_vm.dart (L127-134)	Mutación encadenada in-place del mapa de salida mediante ..addAll().	Propiedades usuarioId, estado y fechaRegistro incorporadas al modelo de dominio ReporteMascota.
9.3	Baja	pantalla_mis_reportes.dart (L630-640)	Mutación in-place mediante ..sort() sobre la lista viva del snapshot.	Lista ordenada independiente generada con List.of().

4.4 Comparativa de código: antes y después

Se muestran cinco casos representativos tomados del informe del agente. En el caso 3 el token se reemplazó por una marca de redacción para no exponer el secreto en este documento.

Caso 1: Extracción de lógica común de fotos e IA (regla 1, DRY)

Archivo: lib/utills/imagen_util.dart frente a reporte_vm.dart y avistamiento_vm.dart

ANTES

```
// En reporte_vm.dart y avistamiento_vm.dart se repetía casi idéntico:
Future<String?> subirFoto(File fotoOriginal) async {
  try {
    final comprimido = await FlutterImageCompress.compressAndGetFile(
      fotoOriginal.absolute.path,
      '${fotoOriginal.path}_comp.jpg',
      quality: 70,
    );
    if (comprimido == null) return null;

    final resultado = await ServicioTFLite.detectarAnimal(File(comprimido.path));
    if (resultado['confianza'] < 0.6) {
      error = "No se detectó un animal con suficiente claridad.";
      return null;
    }

    final uid = FirebaseAuth.instance.currentUser!.uid;
    final ref = FirebaseStorage.instance
      .ref().child('fotos').child(uid)
      .child('${DateTime.now().millisecondsSinceEpoch}.jpg');

    await ref.putFile(File(comprimido.path));
    return await ref.getDownloadURL();
  } catch (e) {
    return null;
  }
}
```

DESPUÉS

```
class ImagenUtil {
  /// Valida mediante IA que la imagen contenga una mascota y la sube a Firebase Storage.
  static Future<String?> validarYSubirFoto({
    required File archivoFoto,
    required String carpetaDestino,
    required String uid,
    double umbralConfianza = 0.6,
  }) async {
    final archivoComprimido = await comprimirImagen(archivoFoto);
    if (archivoComprimido == null) return null;

    final resultado = await ServicioTFLite.detectarAnimal(archivoComprimido);
    final confianza = (resultado['confianza'] as num?).toDouble() ?? 0.0;

    if (confianza < umbralConfianza) {
      throw ValidationException(
        'La imagen no corresponde a una mascota válida '
        '${(confianza * 100).toStringAsFixed(1)}% confianza.',
      );
    }

    final rutaStorage =
      '$carpetaDestino/$uid/${DateTime.now().millisecondsSinceEpoch}.jpg';
    final ref = FirebaseStorage.instance.ref().child(rutaStorage);

    await ref.putFile(archivoComprimido);
    return await ref.getDownloadURL();
  }
}
```

Caso 2: Fail Fast en el registro de avistamientos (regla 3)

Archivo: lib/vistamodelo/reportes/avistamiento_vm.dart

ANTES

```
Future<bool> guardarAvistamiento() async {
  // Peligro: si currentUser es null, la aplicación falla sin contexto
  final uid = FirebaseAuth.instance.currentUser!.uid;

  try {
    _cargando = true;
    notifyListeners();

    final docRef = FirebaseFirestore.instance.collection('avistamientos').doc();
    // Guardado de datos...
    return true;
  } catch (e) {
    _cargando = false;
    notifyListeners();
    return false; // Error silenciado
  }
}
```

DESPUÉS

```
Future<bool> guardarAvistamiento() async {
  // Fail Fast: comprobación estricta de precondition antes de ejecutar lógica
  final uid = FirebaseAuth.instance.currentUser?.uid;
  if (uid == null) {
    throw StateError('No hay usuario autenticado para guardar el avistamiento.');
```

Caso 3: Eliminación de variables globales sueltas (reglas 4 y 6)

Archivo: lib/app.dart

ANTES

```
// Variables libres accesibles y mutables desde cualquier archivo
final String bearer =
  "eyJhbGciOi...[TOKEN REDACTADO EN ESTE INFORME]";
final GlobalKey<NavigatorState> navigatorKey = GlobalKey<NavigatorState>();

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  // ...
}
```

DESPUÉS

```
/// Configuración centralizada de la aplicación.
abstract final class AppConfig {
  /// Token de acceso para el servicio API DNI.
  /// Lee desde variable de entorno si está presente vía --dart-define.
  static const String dniBearerToken = String.fromEnvironment(
    'DNI_TOKEN',
    defaultValue: 'eyJhbGciOi...[TOKEN REDACTADO EN ESTE INFORME]', // ver 0-1
  );
}

/// Servicio que encapsula la clave global de navegación.
abstract final class NavigationService {
  static final GlobalKey<NavigatorState> navigatorKey =
    GlobalKey<NavigatorState>();
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final api = ApiDniServicio(bearerToken: AppConfig.dniBearerToken);
    return MaterialApp(
      navigatorKey: NavigationService.navigatorKey,
      // ...
    );
  }
}
```

Caso 4: Desglose puro de variables (regla 9)

Archivo: lib/modelo/adminusuario_model.dart

ANTES

```
String nombre = '';
String apellido = '';

apellido = (data['apellido'] ?? data['apellidos'] ?? '').toString();
nombre = (data['nombre'] ?? data['nombres'] ?? '').toString();

if (apellido.isEmpty && nombre.isNotEmpty && nombre.contains(' ')) {
  final partes = nombre.trim().split(RegExp(r'\s+'));
  if (partes.length == 2) {
    nombre = partes[0];
    apellido = partes[1];
  } else if (partes.length >= 3) {
    nombre = '${partes[0]} ${partes[1]}';
    apellido = partes[2];
  }
}

if (nombre.isEmpty && data['displayName'] != null) {
  final partes = data['displayName'].toString().split(RegExp(r'\s+'));
  if (partes.length == 2) {
    nombre = partes[0];
    apellido = partes[1];
  }
}
```

DESPUÉS

```
/// Desglosa un nombre completo en nombre y apellido de forma pura.
static (String, String) _desglosarNombreYApellido(String nombreCompleto) {
  final partes = nombreCompleto
    .trim()
```

```
.split(RegExp(r'\s+'))
.where((p) => p.isNotEmpty)
.toList();
if (partes.isEmpty) return ('', '');
if (partes.length == 1) return (partes[0], '');
if (partes.length == 2) return (partes[0], partes[1]);
if (partes.length == 3) return ('${partes[0]} ${partes[1]}', partes[2]);
return ('${partes[0]} ${partes[1]}', partes.sublist(2).join(' '));
}

final String rawApellido =
  (data['apellido'] ?? data['apellidos'] ?? '').toString();
final String rawNombre = (data['nombre'] ?? data['nombres'] ?? '').toString();

final (String nombre, String apellido) = () {
  if (rawApellido.isEmpty && rawNombre.isNotEmpty && rawNombre.contains(' ')) {
    return _desglosarNombreYApellido(rawNombre);
  }
  if (rawNombre.isEmpty && data['displayName'] != null) {
    return _desglosarNombreYApellido(data['displayName'].toString());
  }
  return (rawNombre, rawApellido);
}();
```

Caso 5: Eliminación de mutación de mapas de salida (regla 9)

Archivo: lib/modelo/reporte_mascota.dart y lib/vistamodelo/reportes/reporte_vm.dart

ANTES

```
// En reporte_vm.dart:
await docRef.set(
  reporte.toMap()
  ..addAll({
    'usuarioId': uid,
    'fechaRegistro': FieldValue.serverTimestamp(),
    'estado': 'Perdido',
  })),
);
```

DESPUÉS

```
// En ReporteMascota:
class ReporteMascota {
  String id;
  String usuarioId;
  String estado;
  DateTime? fechaRegistro;
  // ...
}

// En reporte_vm.dart:
reporte.id = docRef.id;
reporte.usuarioId = uid;
reporte.estado = 'Perdido';

await docRef.set({
  ...reporte.toMap(),
  'fechaRegistro': FieldValue.serverTimestamp(),
});
```

4.5 Validación técnica

Se ejecutaron las pruebas existentes y el formateador oficial de Dart. Las tres pruebas

cubren únicamente la validación de avistamiento_vm (descripción vacía, ausencia de foto y ubicación inválida).

```
$ flutter test
00:00 +0: loading test/avistamiento_vm_test.dart
00:00 +0: Validación de avistamiento - Falla si la descripción está vacía
00:00 +1: Validación de avistamiento - Falla si no hay foto
00:00 +2: Validación de avistamiento - Falla si no tiene ubicación válida
00:00 +3: All tests passed!

$ dart format
16 archivos verificados y formateados.
```

4.6 Observaciones del auditor y trabajo pendiente

La revisión crítica de los resultados del agente identificó puntos que no deben darse por resueltos. Se listan con su severidad y la acción recomendada.

ID	Sev.	Observación	Detalle	Recomendación
O-1	Alta	Secreto aún en el código fuente (hallazgos 4.1 y 6.1; Caso 3)	AppConfig.dniBearerToken usa String.fromEnvironment con un defaultValue que contiene el mismo JWT. El secreto sigue en el repositorio y en su historial; --dart-define solo lo sobrescribe si se pasa.	Eliminar el defaultValue, validar al arranque (Fail Fast) que la variable exista, revocar/rotar el token expuesto y purgar el historial de Git.
O-2	Media	debugPrint no desaparece en release (hallazgo 7.2)	debugPrint() solo limita la tasa de salida; en compilaciones release sigue escribiendo en el log del dispositivo. No equivale a un no-op.	Proteger con if (kDebugMode) o usar un logger (dart:developer log / paquete logging) con niveles.
O-3	Media	Singleton estático con setInstancia (hallazgo 6.3)	ServicioTFLite sigue siendo estado estático; setInstancia permite sustituirlo en pruebas, pero no elimina el estado global. ImagenUtil lo invoca de forma estática.	Injectar el servicio por constructor (Provider o get_it) en lugar de acceso estático.
O-4	Media	Cobertura de pruebas limitada	Las 3 pruebas existentes cubren únicamente la validación de avistamiento_vm. «3 de 3 aprobadas» no equivale a «100 % de cobertura». ImagenUtil, AuthWrapper, adminusuario_model y ReporteMascota no tienen pruebas.	Agregar pruebas unitarias, empezando por _desglosarNombreYApellido (función pura) e ImagenUtil con dobles de prueba.
O-5	Media	Cambios de contrato por Fail Fast	validarYSubirFoto ahora lanza ValidationException y guardarAvistamiento puede lanzar StateError; antes devolvían null o fallaban sin contexto. Si la capa de UI no los captura, la excepción llega al usuario sin tratar.	Verificar que cada llamador (pantallas y ViewModels) capture estas excepciones y muestre un mensaje controlado.
O-6	Baja	Clasificación de algunos hallazgos en la regla 5	Los hallazgos 5.1, 5.2 y 5.4 (catch tipado, propagación de errores, herencia de ChangeNotifier) corresponden más a manejo de errores y arquitectura que a nombres. Solo 5.3 y 5.5 son estrictamente de nomenclatura.	Mantenerlos como evidencia, indicando la afinidad con las reglas 3 y 6.

2. Conclusiones

- Se realizó una revisión del código de la aplicación móvil aplicando principios de calidad y buenas prácticas de construcción de software, identificando problemas relacionados principalmente con la duplicación de código, números mágicos, nombres poco descriptivos, variables globales y mensajes innecesarios por consola.
- Se logró reducir la duplicación de código mediante la reutilización de funcionalidades comunes. Entre las principales mejoras se encuentra la creación de FechaUtils para centralizar el formateo de fechas y la implementación del método `_navegarAComentarios()` para evitar repetir el mismo proceso de navegación en diferentes partes de la interfaz.
- La aplicación del principio Fail Fast permitió incorporar validaciones antes de ejecutar operaciones que podrían producir errores, como la comprobación de la existencia de archivos antes de procesarlos y la validación de los datos antes de realizar operaciones de almacenamiento en Firebase.
- Se mejoró la legibilidad y mantenibilidad del código mediante el reemplazo de números mágicos por constantes con nombres descriptivos. Esto permite comprender el propósito de cada valor y facilita su modificación cuando cambien los requerimientos del sistema.
- Se mejoraron los nombres de variables, parámetros y estructuras de datos para que representen claramente su propósito. Esto facilita la comprensión del código y disminuye la posibilidad de errores durante futuras modificaciones.
- Se redujo el uso de variables globales y referencias estáticas compartidas, especialmente en la gestión de comentarios, permitiendo que el acceso a Firestore dependa del identificador y tipo de publicación correspondiente. De esta manera, se obtiene una estructura más organizada y coherente con la jerarquía de datos de la aplicación.
- Se eliminaron mensajes `print()` innecesarios dentro de la lógica de negocio y se mejoró el manejo de errores mediante excepciones y mecanismos apropiados de depuración. Esto permite separar la lógica funcional de los mensajes destinados exclusivamente al diagnóstico del sistema.



- Mediante la auditoría asistida con AntigraVity se identificaron y corrigieron 39 hallazgos distribuidos en las 9 reglas (6 de severidad alta, 17 media y 16 baja), con cambios en 16 archivos del proyecto Flutter.
- La validación técnica consistió en 3 pruebas unitarias aprobadas (3 de 3) y en el formateo de los 16 archivos con dart format. Como esas pruebas no cubren los módulos refactorizados, se recomienda ampliar la cobertura antes de considerar el código plenamente verificado.
- La revisión crítica detectó puntos pendientes que no deben darse por resueltos: el token JWT sigue incrustado como valor por defecto, debugPrint no se elimina en compilaciones release y ServicioTFLite continúa como singleton estático (observaciones O-1 a O-3).
- La revisión asistida por IA resultó útil para detectar y corregir patrones repetitivos, pero sus afirmaciones deben contrastarse con el código y la documentación oficial antes de darse por válidas.